coded

Complete Documentation

A terminal coding agent for any OpenAI-compatible endpoint.

Version 0.4.0

User guide · Skills & plugins · Configuration · Developer guide · Changelog

Contents

1. User Guide

Features

Install

From a clone (what contributors use) · As a tool, isolated from your other packages · Into an existing project or virtualenv · Per-platform notes · Check it worked · Uninstall

Quick start

Configuration

Per-model configuration fields · Self-signed / custom-CA endpoints (TLS) · Built-in provider presets

Usage

Full-screen TUI · Slash commands (in the REPL) · Typing in the REPL · Command suggestions · Permissions · Spending limit · Notifications · Missions: work while you're away · Context & sessions · More folders, excluded paths · Checks after every edit · Jupyter notebooks · Output styles · Your own status line · Testing web apps in a browser · Documents: PDF, Word, Excel, PowerPoint · Repository map · Code intelligence (language servers) · Project memory · Semantic code search · Web search & fetch · Image input (vision) · Checkpoints & undo · Context and exporting · Git & GitHub · Skills · MCP servers · Task list & plan mode · Verification: don't claim success you haven't earned · Hooks · Sandbox · Windows · Background processes · Sub-agents as tasks · Multiple endpoints, models and API keys · Routing by difficulty · Model pools · Large and small models · Outside agents · Worktree isolation · Prompts · Installing skills and plugins · Custom slash commands · Tracing & observability · Prompt-injection guard · Use coded from editors and other agents

Making it faster

Checking your setup

Reviewing pull requests in GitHub Actions

When a model errors

Development

License

2. Writing Skills

Install a skill

How they load (progressive disclosure)

Write your own

3. Writing Plugins

Layout

4. Example Configuration

5. Developer Guide

Project overview

Tech stack

Repository structure

How it fits together

Key conventions

Development workflow

Gotchas

Git & branching

Releasing

Keeping this file accurate

6. Changelog

0.4.0

0.3.0

0.2.0

1. User Guide

Install, configure, and use coded: models, tools, the REPL and TUI, skills, plugins, reviews, missions and more. — from `README.md`

A **Claude Code-style terminal coding agent** that works with any **OpenAI-compatible** model — OpenAI, Groq, DeepSeek, OpenRouter, Together, Fireworks, xAI, Mistral, or local runtimes like Ollama, LM Studio and llama.cpp. Register as many models as you like and switch between them at runtime.

```
┌──────────────────────────────────────────────┐
│ coded — a coding agent for OpenAI-compatible … │
│                                              │
│ model: gpt-4o                                │
│ cwd:   /home/you/project                     │
│                                              │
│ Type your request, or /help for commands.    │
└──────────────────────────────────────────────┘
› add a --verbose flag to the CLI and update the README
```

Features

- **Agentic tool loop** — the model reads, writes, and edits files, searches the codebase, and runs shell commands until your request is done.
- **Missions** — `coded mission "<goal>"`: asks everything up front, works alone on a separate branch, keeps going through outages and step limits, and ends with a report and a notification.
- **Full tool suite** — `read`, `write`, `edit`, `multi_edit`, `apply_patch`, `ls`, `move`, `delete`, `glob`, `grep`, `semantic_search`, `bash` (+ `bash_output` / `kill_shell` for background jobs), `git`, `github`, `web_search`, `web_fetch`, `todo_write`, `enter_plan_mode`/`exit_plan_mode`, `ask_user` (a question with choices), `view_image` (vision models), `skill`, `task` (sub-agents) and `external_agent` (outside agents), plus `list_mcp_resources`/`read_mcp_resource` when an MCP server offers resources, and `lsp` (definitions, references, types, diagnostics from language servers; `"lsp": true`).
- **Task list & plan mode** — the agent tracks multi-step work with `todo_write`; `--plan` / `/plan` keeps it read-only until you approve a plan.
- **Hooks** — run your own commands on `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `SessionStart` and `Stop` (block actions, auto-format, keep the agent going until tests pass).
- **OS sandbox** — `--sandbox` runs `bash` under bubblewrap (Linux) or `sandbox-exec` (macOS): writes limited to the project, no network, no prompts.
- **Parallel tools** — independent reads/searches requested together run concurrently.
- **Many endpoints, models & keys** — per-model endpoint/key, automatic retry with backoff, **fallback** to another endpoint/model, **per-role models** (cheap sub-agents/compaction, strong reviews), and `coded models --check`.
- **Reasoning models** — `reasoning_effort`, provider `extra_body`, and streamed thinking (`--show-reasoning`).
- **Outside agents** — delegate to Claude Code, Codex, Gemini CLI, Aider, OpenCode, Cursor or Goose when they're installed, or any CLI you configure.

- **Worktree isolation** — sub-agents and outside agents can work in their own git worktree/branch, so parallel edits never collide.
- **Custom slash commands** — Markdown prompt templates in `.coded/commands/`.
- **Editable prompts** — the built-in prompts are eight Markdown files (`system`, `coder`, `tools`, `security`, `skills`, `planner`, `reviewer`, `debugger`) that you can override per user or per project (`coded prompts`).
- **Skill & plugin installs** — `coded skill install` and `coded plugin install` from a folder, a `.zip` (path or URL), a git URL or GitHub `owner/repo`; plugins bundle skills, commands, hooks, MCP servers and outside agents; marketplaces; Claude Code plugins install as-is.
- **Tracing** — JSONL spans for turns, LLM calls and tools (`--trace`), plus optional OpenTelemetry export.
- **Prompt-injection guard** — web/PR/MCP output is fenced as untrusted, and risky actions after reading it are flagged (or always confirmed, in strict mode).
- **Project trust** — a repository's own `.coded/config.json` cannot run hooks or MCP servers, point a model at another endpoint, or loosen permissions until you trust it: coded asks once at start and lists every command and endpoint (again if they change). Non-interactive runs ignore those settings unless `--trust-project` or `CODED_TRUST_PROJECT=1`. Reading or writing outside the project, and `web_fetch`/`browser` requests to localhost, the local network or `file://`, ask first.
- **Self-healing** — `/heal` (or `"verify": {"auto": true}`) runs the project's own tests after each change — detected: npm, cargo, go, pytest, make — and sends failures back to fix; the affected tests run first, an identical failure twice stops the loop, and it asks before running the repo's code.
- **Dependency map** — `repo_map` answers `imports`, `dependents`, `impact` (what a change can affect, and which tests to run) and `cycles` for Python and JS/TS.
- **Command Center** — `/dashboard` (also mid-turn): agents and sub-agents, per-model cost, per-tool calls/failures/denials/time, checks and recent problems.
- **Time-travel** — `/timeline` replays each request step by step with outcomes and the files it changed; `/timeline restore N` previews, then goes back.
- **Adversarial reviewer** — `/redteam` or `coded review --adversarial`: a read-only agent tries to break the change; findings need a file:line and a trigger.
- **Project twin** — `/twin`: languages, modules, central files, cycles, hot spots and what changed since you last looked, from a per-file cache.
- **Self-improvement lab** — `coded lab`: recurring failures across past sessions, with proposals you approve one by one (`--apply`).
- **Refactor finder** — `/refactor [path]`: a scan for long, branchy or deeply nested functions and copy-pasted blocks (`repo_map health`), then small, tested, behavior-preserving fixes by the built-in `refactor` skill.
- **Editor & agent integration** — `coded acp` speaks the Agent Client Protocol (Zed, JetBrains, …); `coded mcp-serve` exposes coded as an MCP server.
- **Semantic code search** — build an embedding index (`coded index`) and let the agent search the codebase by meaning, not just literal text.
- **Web search & fetch** — `web_search` (Tavily/Brave/SerpAPI/SearXNG) and `web_fetch` to read pages, for up-to-date research.
- **Image input** — attach images for vision-capable models (`--image`, `/image`).
- **Checkpoint / undo** — file edits are snapshotted per turn; `/undo` reverts them.
- **Git & GitHub tools** — a `git` tool (read-only runs freely, mutations are gated) and a `github` tool (list/get/create PRs, comment) via `GITHUB_TOKEN`.

- **Any OpenAI-compatible endpoint** — configurable base URL, API key, and model id, with built-in presets for common providers.
- **Multiple models, switchable** — define models in config, switch live with `/model <name>`.
- **Permission layer** — file writes and shell commands ask for approval (once / always / deny), or run unattended with `--yolo`.
- **Two front-ends** — a modern inline **REPL** (streaming Markdown, live status bar, ghost-text history suggestions, rich completion, framed permission/diff prompts) and an optional **full-screen TUI** (`coded --tui`) with a scrolling transcript, tool calls and numbered diffs, a status line, and a permission modal.
- **Sub-agents** — the `task` tool spawns a focused agent with its own context for large searches or multi-step subtasks.
- **Skills & agent roles** — Claude Code-style `SKILL.md` instruction packs with progressive disclosure; the agent auto-invokes them when a task matches. Ships with built-in `planner`, `reviewer`, `debugger`, `qa`, `security` and `refactor` roles.
- **MCP client** — connect Model Context Protocol servers and expose their tools to the agent (optional).
- **Context auto-compaction** — long sessions are summarized automatically as they approach the model's context window (`/compact` to force it).
- **Session save/resume** — conversations persist to disk; `--continue` / `--resume <id>`, or `/resume` inside coded, pick up where you left off.
- **Permission rules** — allow/deny tool actions by pattern in config, plus a colored diff preview before every `write`/`edit`.
- **Cost tracking** — `/cost` shows token usage and an estimated dollar cost; `--print --output-format json` emits machine-readable results for scripts/CI.
- **Project instructions** — `AGENTS.md` / `CODED.md` / `CLAUDE.md` files are loaded from `~/.config/coded/`, the repo root and every directory down to the working directory; files in subdirectories load when the agent first works there.

Install

coded is a normal Python package (3.10+) with a `coded` console script. It runs on Linux, macOS and Windows — see Windows for what differs there. Pick whichever of these fits your machine; they all end with `coded --version` working.

From a clone (what contributors use)

```
git clone https://github.com/krishnavams/coded
cd coded
python3 -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -e .                  # core
pip install -e '.[mcp]'           # + MCP client     (cmd.exe: use ".[mcp]")
pip install -e '.[tui]'           # + full-screen UI
pip install -e '.[dev]'           # + pytest
pip install -e '.[mcp,tui,dev]'   # everything
coded --version
```

`-e` (editable) means your edits take effect without reinstalling.

As a tool, isolated from your other packages

Best for daily use — one command, its own environment, on your PATH:

```
pipx install git+https://github.com/krishnavams/coded          # pipx install pipx first
pipx install "coded[mcp,tui] @ git+https://github.com/krishnavams/coded"
pipx upgrade coded

uv tool install git+https://github.com/krishnavams/coded       # or with uv
uvx --from git+https://github.com/krishnavams/coded coded      # run once, install nothing
```

Into an existing project or virtualenv

```
pip install git+https://github.com/krishnavams/coded
uv pip install git+https://github.com/krishnavams/coded        # uv
poetry add git+https://github.com/krishnavams/coded            # poetry
```

Add a branch or tag with `@name`, e.g. `…/coded@main`. While the repository is private, `git+https://…` needs credentials — use SSH (`git+ssh://git@github.com/krishnavams/coded`) or a token, or clone first and install from the directory.

Per-platform notes

Where	Notes
Linux	Install `python3-venv` / `python3-pip` from your package manager first; for the sandbox, `bubblewrap`. Most distributions now block `pip install` outside a virtualenv (PEP 668) — use a venv, `pipx`, or `uv`.
macOS	`brew install python@3.12 pipx`. The sandbox (`sandbox-exec`) is built in.
Windows	Install Python from python.org or `winget install Python.Python.3.12`, then use PowerShell or Windows Terminal. Install Git for Windows too: coded uses its Git Bash for the `bash` tool. In `cmd.exe` quote extras with double quotes: `pip install -e ".[mcp]"`.
WSL	Treat it as Linux; it also gives you the sandbox, which native Windows has no backend for.
Docker	`python:3.12-slim` has no git, so install it first: `RUN apt-get update && apt-get install -y --no-install-recommends git && rm -rf /var/lib/apt/lists/*`, then `RUN pip install git+https://github.com/krishnavams/coded`. (Copying the repo in and `pip install .` needs no git.) Pass keys with `-e OPENAI_API_KEY`, mount your project at `/work`, run `coded --cwd /work`. Use `--yolo` only in a throwaway container.
CI (GitHub Actions)	`pip install git+https://github.com/krishnavams/coded`, then `coded review --fail-on NEEDS_CHANGES` with the key in a repository secret. See `.github/workflows/coded-review.yml`.
Remote box / server	Same as Linux. For a long session use `tmux`; one-shot work needs no terminal at all: `coded -p "…" --output-format json`.
No internet on the target machine	On a connected machine with the same Python version and OS: `pip wheel -w wheels .` (coded plus its dependencies, ~24 wheels). Copy `wheels/` over, then `pip install --no-index --find-links wheels coded`. Point `base_url` at your internal endpoint.

Check it worked

```
coded --version
coded models            # configured models, pools and roles (empty is fine at first)
coded models --check    # sends a tiny request to each endpoint
coded config init       # writes ~/.config/coded/config.json
```

Nothing is needed beyond a key or a local endpoint; see Configuration.

Uninstall

```
pip uninstall coded          # or: pipx uninstall coded / uv tool uninstall coded
rm -rf ~/.config/coded       # settings, skills, plugins, saved sessions
```

Quick start

The fastest path — just set an API key and go:

```
export OPENAI_API_KEY=sk-...
coded                        # start the interactive REPL
coded "explain what this repo does"   # start with an initial prompt
```

Point it at **any** OpenAI-compatible endpoint without a config file:

```
# A local Ollama model:
coded --provider ollama --model-name qwen2.5-coder:7b

# Any custom endpoint:
coded --base-url http://localhost:8000/v1 --api-key sk-x --model-name my-model

# Groq:
export GROQ_API_KEY=...
coded --provider groq --model-name llama-3.3-70b-versatile
```

Configuration

Create a config file with your models:

```
coded config setup         # interactive: add, test and wire up models
coded config init          # writes a sample ~/.config/coded/config.json
coded config path          # print the path
coded config show          # show the effective, merged config
```

`coded config setup` (also `/setup` inside the REPL) is the quickest way in. It asks for the endpoint once, asks the endpoint which models it serves, and lets you pick several by number; then it names them, sends a test request, and sets the default model, per-role models, pools and difficulty routing. Keys default to an **environment variable name** — a literal key is read without echo, never printed back, and written only to your user config with mode 600. Nothing is written until you choose "save and quit", the file is replaced atomically, and keys the wizard doesn't know about (hooks, MCP servers, per-model `limits`) are left exactly as they were.

Config is layered (later overrides earlier):

1. Built-in provider presets
2. `~/.config/coded/config.json` (or `$CODED_CONFIG`)
3. `./.coded/config.json` (per-project)
4. Command-line flags

A model entry only needs a provider and a model id; the base URL and API-key environment variable are inherited from the provider preset. Example:

```
{
  "default_model": "gpt-4o",
  "models": {
    "gpt-4o":     { "provider": "openai",   "model": "gpt-4o" },
    "fast":       { "provider": "groq",     "model": "llama-3.3-70b-versatile" },
    "local":      { "provider": "ollama",   "model": "qwen2.5-coder:7b" }
  }
}
```

Minimal — just a URL and a model name. If your endpoint is OpenAI-compatible, an entry can be as short as a `base_url` and a `model`. No provider or API key is needed for self-hosted/keyless endpoints (coded falls back to a placeholder key); add `api_key` / `api_key_env` if yours requires one:

```
{
  "default_model": "my-model",
  "models": {
    "my-model":   { "base_url": "http://localhost:8000/v1", "model": "my-model-name" },
    "remote":     { "base_url": "https://my-host/v1", "model": "another-model", "api_key_env":
"MY_KEY" }
  }
}
```

Or add one for a single run without any config file:

```
coded --base-url http://localhost:8000/v1 --model-name my-model-name
```

Have a lot of models? `config.example.json` is a ready-to-trim catalogue with ~25 models across every provider, each with its own context window, token limit, and pricing filled in. Copy it to your config path and delete the ones you don't use:

```
cp config.example.json ~/.config/coded/config.json
```

Per-model configuration fields

Each entry under `models` accepts:

Field	Required	Description
`provider`	yes*	Provider preset name (supplies `base_url` + key env).
`model`	yes	Concrete API model id sent to the endpoint. Defaults to the alias.
`base_url`	no	Override the provider's base URL (required for `openai-compatible`).

Field	Required	Description
`api_key`	no	Inline key (avoid committing real keys).
`api_key_env`	no	Env var holding the key (overrides the provider default).
`max_tokens`	no	Max output tokens per response.
`temperature`	no	Sampling temperature. Omit for models that reject it (e.g. some reasoning models).
`context_window`	no	Total context size (informational; default 128000).
`input_cost`	no	USD per **million** input tokens (used by `/cost`).
`output_cost`	no	USD per **million** output tokens.
`cached_input_cost`	no	USD per million input tokens served from the provider's prompt cache (most providers bill these cheaper). Unset: priced like `input_cost`. `/cost` shows how many input tokens were cached.
`supports_tools`	no	Set `false` for models without function-calling (e.g. search-only models); the agent then runs without tools.
`extra_headers`	no	Extra HTTP headers (e.g. OpenRouter's `HTTP-Referer`/`X-Title`).

* Instead of `provider`, you may set `base_url` + `api_key`/`api_key_env` directly.

| `verify_ssl` | no | Set `false` to disable TLS certificate verification (insecure; self-signed endpoints). | | `ca_bundle` | no | Path to a custom CA cert/bundle (PEM) to trust for this endpoint. |

Self-signed / custom-CA endpoints (TLS)

If your OpenAI-compatible endpoint uses a private CA or a self-signed certificate, either **trust a CA bundle** (recommended) or **disable verification** (insecure). Per model in config:

```
{
  "models": {
    "corp":   { "base_url": "https://llm.internal/v1", "model": "x", "ca_bundle":
"/etc/ssl/corp-ca.pem" },
    "selfsigned": { "base_url": "https://192.168.1.5/v1", "model": "x", "verify_ssl": false }
  }
}
```

Or from the CLI for a one-off:

```
coded --base-url https://llm.internal/v1 --model-name x --ca-bundle /etc/ssl/corp-ca.pem
coded --base-url https://192.168.1.5/v1  --model-name x --insecure   # skips verification
(insecure)
```

`--ca-bundle` / `--insecure` also work on `coded review` and `coded commit`, and a `CODED_CA_BUNDLE` environment variable is used as a default CA bundle when set. Disabling verification prints an

insecure-connection warning.

Built-in provider presets

`openai`, `anthropic`, `google` (Gemini), `openrouter`, `groq`, `together`, `deepseek`, `mistral`, `xai`, `fireworks`, `cerebras`, `perplexity`, `nvidia`, `deepinfra`, `moonshot`, `ollama`, `lmstudio`, `llamacpp`, `vllm`, `jan`, and `openai-compatible` (supply your own `base_url`). Each preset knows the default base URL and the environment variable that holds the API key (e.g. `GROQ_API_KEY`, `ANTHROPIC_API_KEY`). `anthropic` and `google` use those vendors' OpenAI-compatible endpoints.

Usage

```
coded [PROMPT]                 # REPL, optionally seeded with a prompt
coded --tui                    # full-screen terminal UI (pip install 'coded[tui]')
coded -p "run the tests"       # non-interactive: run once and print
coded -m fast "..."            # pick a configured model alias
coded --yolo "..."             # auto-approve all tool actions
coded --no-mcp                 # disable MCP servers for this run
coded --continue               # resume the most recent session here
coded --resume <id>            # resume a specific session
coded -p --output-format json "..."   # machine-readable result
coded -p --output-format stream-json "..."  # one JSON event per line as it works
coded models                   # list configured models
coded index                    # build the semantic search index
coded review [target]          # non-interactive review pipeline (CI); exit 1 on NEEDS_CHANGES
coded commit [--commit]        # draft a Conventional Commits message from staged changes
coded doctor [--network]       # check install, config, keys, tools (and endpoints)
coded sessions                 # list saved sessions
```

Full-screen TUI

`coded --tui` launches a Textual app: a scrolling transcript (user turns, streamed assistant Markdown, tool calls shown as `● Edit(app.py)` with their results, line-numbered diffs), the task list while the agent has one (docked right above the input, so progress stays in view), an input box with a **status line** under it (model, mode, turns, tokens, cost), and a **permission modal** for gated actions. Keys: `Enter` send, `Ctrl+B` side panel, `Ctrl+L` clear, `Ctrl+U` undo edits, `Ctrl+C` quit.

A **side panel** on the right shows, each part only when it has something: the session (model, plan mode, how full the context window is and where it compacts, turns/tokens/cost), the files changed this turn with `+added −removed` line counts, messages queued while the agent works, and background shells with their state. It opens by itself when the terminal is at least 110 columns wide; `Ctrl+B` opens or closes it at any width. Files changed by `bash` commands are not listed (they are not tracked for `/undo` either). It reuses the same agent, tools, permissions, and session persistence as the REPL. Install the extra:

```
pip install -e '.[tui]'      # adds Textual
```

Slash commands (in the REPL)

Command	Description
`/help`	List commands

Command	Description
`/model [name]`	Show or switch the active model
`/models`	List configured models
`/setup`	Configure models interactively (then reloads the session)
`/tools`	List available tools
`/skills`	List available skills
`/skill <name> [task]`	Invoke a skill now (or just `/<name> [task]`)
`/review [target]`	Run reviewer→qa→security sub-agents and aggregate
`/image <path>... [prompt]`	Attach image(s) for a vision model
`/paste`	Attach the image on the clipboard (also **Ctrl+V**)
`/vim`	Toggle vi key bindings in the input
`/undo`	Revert the last turn's file changes
`/rewind [n] [--keep-files]`	Go back to before request *n*: conversation and its file changes
`/resume [n | id]`	List this folder's saved conversations, or reopen one (the current one stays saved)
`/context`	What fills the context window
`/add-dir [path]`	Let the agent work in another folder too
`/style [name|off]`	List or switch output styles
`/export [path]`	Save the conversation as Markdown
`/checkpoints`	List saved checkpoints
`/compact`	Summarize the conversation to free context
`/init`	Analyze the repo and write a `CODED.md` guide
`/cost`	Token usage and estimated cost
`/clear`, `/reset`	Clear the conversation
`/config`	Show loaded config files / MCP servers
`/plan [request]`	Toggle plan mode (read-only until a plan is approved)
`/todos`	Show the agent's task list
`/shells [kill <id>]`	List or stop background shells
`/mission <goal>`	Start an unattended mission with this session's model (`/mission --resume`, `/mission --list`)
`/btw <question>`	A quick side question, answered from the conversation so far (no tools). Works **while the agent is working**, without interrupting it, and neither the question nor the answer is added to the conversation

Command	Description
`/tasks [id | stop <id>]`	Sub-agents and background shells: list, show one's steps and report, or stop one (also **Ctrl+T**)
`/agents`	List outside agents available for delegation
`/commands`	List custom slash commands
`/plugins`	List installed plugins
`/prompts`	Show which prompt files are in use
`/<custom> [args]`	Run a custom command from `.coded/commands/`
`/permissions [...]`	Show or change the mode and allow/deny rules; `save` keeps them
`/budget [dollars|off]`	Show or set the spending limit
`/exit`, `/quit`	Quit

Typing in the REPL

Key / syntax	What it does
`@path`	Attach a file to the message: `explain @src/app.py`, just some lines with `@app.py#L10-40`, a folder listing with `@src/`, an image with `@shot.png`; quote paths with spaces (`@"my notes.md"`). **Tab** completes paths. Works in the TUI and `coded -p` too.
Alt+Enter, **Ctrl+J**, or `\` then Enter	New line without sending
←/→, **Home**/**End**, **Delete**, **Ctrl**/**Option**+**←/→**, **Shift**+any of them	Move, delete and select anywhere in the message — also in the input box shown while the agent works (see below)
Esc	Stop the agent mid-turn (Ctrl-C works too); at a permission question, Esc means "no"
Shift+Tab	Cycle mode: normal → accept edits → plan. While a turn is running: accept edits on/off, from the next edit on
Ctrl+G	Write the message in `$EDITOR`
Ctrl+V	Attach the clipboard's image (macOS: built in, faster with `pngpaste`; Linux: needs `wl-clipboard` or `xclip`; Windows: PowerShell)
Ctrl+R	Search earlier messages
↑ (during a turn)	Take the newest queued message back into the input to edit it; Enter queues it again (↑ again for the one before). Also in the TUI
Ctrl+B	While a sub-agent runs: move it to the background and keep chatting; its report reaches the agent when it finishes
Ctrl+O	Expand / collapse tool output: shows the last shortened result (`… +13 lines`) in full, and every step of a running sub-agent
Ctrl+T	List running and finished tasks (`/tasks`)

Set `"editing_mode": "vi"` in your config for vi key bindings (or toggle with `/vim`).

Command suggestions

Type `/` and a dropdown lists the commands — the same in the REPL prompt, in the input box while the agent works, and in the TUI. It completes as you go:

- **Commands, subcommands, flags and values**: `/permissions` offers `allow deny remove mode save`, `/permissions mode` the three modes, `/mission --` its flags, `/model` your configured models and pools, `/style` the output styles, `/resume` this folder's saved sessions, `/rewind` and `/timeline` this conversation's requests, `/tasks stop` what is running, `/memory forget` the notes, `/permissions allow` a rule for each tool (and `bash(<cmd> *)` for the commands the agent ran), `/review`, `/export`, `/add-dir`, `/image` files and folders, `@` any path. Custom commands show their `argument-hint`, plugins and skills their source.
- **Forgiving**: prefix, letters in order (`/rvw` → `/review`) and typos (`/reveiw` → `/review`). An unknown command answers "Did you mean …?".
- **Ranked for you**: commands you use in this project come first, and whole lines you ran before come back (`/model fast`). Learned per project in `~/.config/coded/command_usage.json`.
- **What next**: a bare `/` puts first what the session points at — failed checks → `/heal`, `/timeline`; edits → `/review`, `/undo`; sub-agents running → `/tasks`; a full context window → `/compact`; a call you denied → `/permissions` — each with the reason.

Keys: ↑/↓ choose, **Tab** takes the highlighted (or first) row, **Enter** takes a row you moved to (otherwise it sends the line), **Esc** closes the list. Taking a row only edits the line — nothing runs until you press Enter. During a turn, ↑ and Esc keep their usual meaning (edit queued message, stop) whenever the list is closed. The TUI's list can be clicked; in the REPL set `"mouse": true` to click it too (off by default: while the prompt owns the mouse, your terminal's own text selection does not work there).

Permissions

Tools that change files (`write`, `edit`, `move`, `delete`) or run commands (`bash`, `git` mutations, `github` writes, MCP tools) ask before running. Answer `y` (once), `a` (always, for this session), or `n` (deny). Before a `write`/`edit` you see a colored **diff preview** of the change. Use `--yolo` / `--auto-approve` to skip prompts, or set `"auto_approve": true` in config. In non-interactive `--print` mode with no auto-approve, such actions are denied by default (safe).

Allow/deny rules. Cut prompt fatigue by pre-approving (or blocking) actions by pattern. Patterns are `tool` or `tool(glob)` matched against the command/path; **deny always wins**, even over `--yolo`:

```
{
  "permissions": {
    "allow": ["bash(git *)", "bash(pytest*)", "read", "grep"],
    "deny":  ["bash(rm -rf*)", "bash(curl*)"]
  }
}
```

Modes. Shift+Tab (REPL and TUI) cycles three modes, shown in the status bar: *normal* (ask before changes), ⏭ **accept edits** (file edits — `write`, `edit`, `multi_edit`, `apply_patch`, `move` — run without

asking; deleting files and running commands still ask) and ⏸ **plan** (read-only). Start in accept-edits mode with `--accept-edits` or `"accept_edits": true`.

`/permissions` shows the mode and every rule, and changes them for the session: `/permissions allow bash(npm test*)`, `/permissions deny bash(curl*)`, `/permissions remove <rule>`, `/permissions mode accept-edits`. `/permissions save` writes the current rules to the project's `.coded/config.json`.

Spending limit

Stop a session before it costs more than you meant to spend:

```
coded --max-cost 2.00           # dollars, from the models' input_cost/output_cost
```

or `"max_cost": 2.0` in config. The turn stops with a note once the session's estimated cost reaches the limit; `/budget 5` raises it, `/budget off` removes it, and `/budget` shows what has been spent.

Notifications

When work that ran longer than `notify_after` seconds (default 30) ends, coded tells you how: **Task completed**, or **Task stopped** with the reason — you stopped it, an error, the step limit, it was not making progress, the spending limit. It also calls you over when a long request waits for your answer. Choose the channels with `"notify"` (one or several, comma-separated):

Channel	What you get
`desktop`	A notification banner (osascript on macOS, `notify-send` on Linux; else the bell)
`voice`	Said aloud (`say` on macOS, `spd-say`/`espeak` on Linux, PowerShell speech on Windows)
`bell`	The terminal bell (the default)
`line`	`✓ Task completed (2m 05s) — …` / `■ Task stopped: … — …` in the transcript

`"notify": "all"` uses every channel, `"off"` none. In the REPL or TUI, `/notify off` and `/notify on` switch it for the session (also while the agent works), `/notify desktop voice` picks channels, `/notify test` sends one now. A mission always announces how it ended, however long it ran (`coded mission … --notify desktop,voice`).

Missions: work while you're away

```
coded mission "add CSV export to the reports page" --budget 2.00
coded mission "fix the flaky date tests" --notify-url https://ntfy.sh/my-topic
```

Inside coded, `/mission <goal>` (also `/mission --resume`, `/mission --list`) does the same with the session's model and folder.

A mission asks everything up front — trust this project? run its checks (`npm test`, `pytest`, …) after each change? run shell commands without asking? — and then works alone until the goal is done:

- **On a copy.** It runs in a new git worktree on a `coded/mission-…` branch; your checkout is never touched (`--here` works in place instead).
- **Plans first.** The first run may only look around: every change is refused until it has written a task list (it is asked twice; without one the work starts anyway and the report says so). Then it

works through the list, and the project's checks gate every change. Nothing waits for you: an action nobody approved is refused, and a question becomes a stated assumption.
- **Keeps going.** A model outage is retried after a pause, a run that hit the step limit or left tasks open is sent back to finish (up to `--max-restarts`, default 3). It stops when it is clearly stuck, the `--budget` runs out, or you press Ctrl-C.
- **Steer it if you are watching.** On a terminal the input box stays at the bottom while it works, with what it is doing (`◆ running tool bash · 1 follow-up waiting`). Type a message and it reaches the mission at its next step ("use port 8080", "skip the admin pages"), even across its restarts; `/btw`, `/dashboard`, `/todos`, `/cost` and `/help` answer at once (other commands cannot run during a mission: they go back into the box). **Esc** stops it like Ctrl-C. The report lists what you told it, and anything typed too late to be delivered.
- **Survives being killed.** Every step is saved. If the computer sleeps or restarts or the terminal closes, `coded mission --resume` (in the project or the worktree) carries on with the same answers, branch and task list, and still ends with the report. `coded mission --list` shows the missions.
- **Ends with a report** in `.coded/missions/<time>.md`: the result, changed files, checks, tasks, cost, what went wrong on the way, the agent's summary and the exact commands to review, merge or discard the branch. The changes are committed on the branch. You get a desktop notification, and with `--notify-url` the report is POSTed there too (an ntfy.sh topic shows it on your phone).

Without a terminal (`-y`/`--yes`, cron, CI) nothing is asked: unapproved checks and shell commands stay off unless `--allow-bash` (or `--yolo`) says otherwise. Deny rules always win.

Context & sessions

Auto-compaction is **on by default**: when the conversation reaches 80% of the model's `context_window` (`compact_ratio`), the older messages are summarized in place and coded says so ("Summarized earlier conversation to fit context"). The task list survives it, and the cut is made at a user-message boundary so a tool call is never split from its result. `/compact` forces it; `{"auto_compact": false}` turns it off. The threshold follows whichever model answers, so it shrinks if a smaller fallback takes over — a model with no `context_window` set is assumed to be 128k.

Long conversations are **auto-compacted**: as the estimated input nears the model's `context_window`, older messages are summarized into a briefing while a recent, consistent tail is kept verbatim. Force it with `/compact`; disable with `--no-compact` or `"auto_compact": false`.

Sessions **persist to disk** (`~/.config/coded/sessions/`, autosaved each turn):

```
coded --continue        # resume the most recent session for this directory
coded --resume <id>     # resume a specific one
coded sessions          # list them
```

Inside coded, `/resume` lists this folder's other saved conversations (newest first, with their first request) and `/resume 2` — or `/resume <id>`, or an id prefix — switches to one without restarting. The conversation you leave is saved first, and the one you open keeps saving under its own id. File changes are not touched: `/undo` still reverts this run's edits.

More folders, excluded paths

`--add-dir <path>` (repeatable), `"add_dirs": [...]` in config, or `/add-dir <path>` in the REPL lets the agent work in folders besides the current one — a shared library next to the app, a backend and a frontend in separate repos. The agent is told about them, `@mentions` find files there, and with the sandbox on they are writable too.

`.codedignore` in the project root lists paths the agent must never read or change, in `.gitignore` syntax:

```
secrets/
*.pem
/build
!public.pem
```

The file tools refuse them, `grep`/`glob`/`ls` leave them out, and `@mentions` won't attach them. A shell command can still reach them, so add a deny rule (e.g. `"bash(*secrets*)"`) where that matters.

Checks after every edit

`"diagnostics": true` runs a quick check on each file right after the agent writes or edits it and, if it fails, puts the error in front of the model in the same step — so a syntax error or a type error is fixed before it moves on. The built-in checks use what is installed: Python (`ruff`, else `python -m py_compile`), JSON, shell scripts (`bash -n`) and JavaScript (`node --check`). Add your own per file pattern, `{file}` being the edited file:

```
"diagnostics": {
  "commands": {"*.py": "mypy {file}", "*.ts": "npx tsc --noEmit -p ."},
  "builtin": true,
  "timeout": 20
}
```

Jupyter notebooks

The `read` tool shows a notebook cell by cell (`cell 3 [code] id=…`) with its text outputs; `pages: "3-5"` picks cells. The agent edits notebooks with `notebook_edit` — replace, insert or delete one cell — which keeps the JSON valid and clears a changed cell's stale outputs.

Output styles

`/style` lists styles and switches how the agent talks: **explanatory** (explains why it chose an approach and shares insights about your code), **concise** (results only, no preamble or recap) and **learning** (does the scaffolding and leaves small, key pieces for you to write). `/style off` returns to the default; `"output_style": "concise"` sets one in config. Add your own as `.coded/styles/<name>.md` (or `~/.config/coded/styles/`): a `description` in the frontmatter and the instructions as the body.

Your own status line

Replace the status line under the input with your own script's output:

```
"statusline": {"command": "~/bin/coded-status", "refresh": 5}
```

The command gets a JSON object on stdin — `model`, `cwd`, `mode`, `turns`, `input_tokens`, `output_tokens`, `cost`, `tasks` — and the first line it prints (ANSI colors work) is shown in the REPL and the TUI. It runs in the background at most every `refresh` seconds; while it hasn't answered, or if it fails, the built-in line is shown. For example, the git branch and the cost:

```
#!/bin/sh
cost=$(python3 -c 'import json,sys; print(json.load(sys.stdin)["cost"])')
echo "⎇ $(git branch --show-current 2>/dev/null)  ·  \$$cost"
```

Testing web apps in a browser

With `"browser": true` the agent gets a `browser` tool that opens a page in headless Chromium, runs steps (click, fill, press, select, wait), and reports the rendered text, JavaScript console errors, failed requests and a screenshot (saved under `.coded/screenshots/`). Loading a page runs without asking; steps that act on the page ask first. Install it with `pip install 'coded[browser]' && playwright install chromium`, or use a Chrome/Chromium you already have: `"browser": {"executable_path": "/usr/bin/chromium"}`.

Documents: PDF, Word, Excel, PowerPoint

```
pip install 'coded[docs]'      # pypdf, openpyxl, python-docx, python-pptx
```

With the extra installed, the `read` tool reads documents as text, paged through with the same `offset`/`limit` and per-model budgets as source files:

File	Read as
`.pdf`	text per page (`===== page 3 =====`)
`.docx`	headings, paragraphs and tables in document order
`.xlsx`	each sheet as numbered rows of cell values
`.pptx`	per slide — title, bullets, tables and speaker notes

`pages` picks part of a document first: `"3"`, `"2-5"`, `"1,4,9"`, or a sheet name. Spreadsheets show the values Excel last calculated; a formula that was never calculated (a file written by a script) is shown as the formula rather than a blank.

Out of scope, and reported as such rather than guessed at: **scanned PDFs** (pages that are images need OCR — the tool says so), charts and pictures inside Office files, password-protected files, and the pre-2007 binary formats (`.doc`, `.xls`, `.ppt` — the error gives the `soffice` command to convert them). Without the extra, reading a document fails with the exact `pip install` line; it never falls back to decoding a binary file as text.

Repository map

```
repo_map                      # the whole repo, ranked
repo_map path=src/api         # one subtree
repo_map focus=PermissionManager
```

`repo_map` is an outline of the codebase — each file's classes, functions and types — with the files **the rest of the code depends on most** first. Grep answers "where does this string appear"; this answers "what is this made of, and which part matters", which is the question a model with a small context window has to answer before it can act.

Ranking is PageRank over a graph whose edges are "this file names a symbol that file defines", so a small, widely imported module outranks a large one nobody references. Method names (`run`, `save`) and names defined in many files are excluded from the graph — otherwise whichever file has the most of them looks like the heart of the project.

Extraction is Python's own `ast` plus conservative per-language regexes (Go, Rust, JS/TS, Java/Kotlin, Ruby, PHP, C/C++, Swift, Lua, shell) — no tree-sitter and no extra dependency, so it is an *outline*, not a parse tree. It lists definitions only, never code, and everything is bounded: files scanned, bytes per file, wall time, output. Binary, minified and generated files are skipped.

Code intelligence (language servers)

With `"lsp": true` in the config the agent gets an `lsp` tool backed by the language servers you have installed — the same engine your editor uses — and can ask for a symbol's **definition**, **type definition**, **implementations**, all its **references**, its **hover** (type, signature, docs), a file's **outline**, a **project-wide symbol search**, and a file's **diagnostics** (type errors and warnings) after an edit. It names the symbol and its line; coded works out the column.

Servers are found on PATH and started on first use, one per language, with the project as the workspace:

Language	Servers tried, in order
Python	`pyright-langserver`, `basedpyright-langserver`, `pylsp`, `jedi-language-server`
TypeScript / JavaScript	`typescript-language-server`
Go · Rust · C/C++	`gopls` · `rust-analyzer` · `clangd`
Java · Ruby · PHP · Lua · C#	`jdtls` · `ruby-lsp` / `solargraph` · `intelephense` · `lua-language-server` · `csharp-ls`

Pick a server, add a language or turn one off:

```
{
  "lsp": {
    "python": ["pylsp"],
    "zig": {"command": ["zls"], "extensions": [".zig"]},
    "rust": false,
    "timeout": 60
  }
}
```

`coded doctor` shows which servers it finds. Without a server for a file, the tool says so and the agent falls back to `grep` / `repo_map`.

Two things to know. A language server is a program that analyses the project, and some run the project's own build steps while doing so (rust-analyzer runs build scripts and macros, gopls runs `go`

commands), outside coded's sandbox and without a permission prompt — turn `lsp` on for code you trust. And results from files excluded by `.codedignore` show the location only, never the file's text.

Project memory

```
/memory                                  # list
/memory add gotcha: the e2e suite needs Docker
/memory forget m3
```

What a session learns about a repository normally dies with `/clear`. Project memory keeps a few short notes — architecture, conventions, decisions, fixes, gotchas, facts — in `./.coded/memory.json` (gitignored), and puts them in the system prompt at the start of every future session in that project, labelled as recorded observations that may be out of date, never as instructions.

The agent can add notes with the `memory` tool, but every write **asks you first and shows the exact text**, and a turn with untrusted content in context (web, GitHub or MCP output) cannot write at all — a page should not be able to plant a durable instruction. The model can only ever write the *project* store, never your user-level config, so one hostile repository can't follow you elsewhere. The store is capped, your own notes outlive the agent's, and `/memory` is the authority: list, add, forget, clear.

Semantic code search

Build an embedding index of the repo, then the agent's `semantic_search` tool finds code by meaning. Configure an embedding model, then index:

```
{
  "embedding": { "provider": "openai", "model": "text-embedding-3-small" }
}
```

```
coded index          # build/refresh .coded/index/index.json
coded "where is auth handled?"   # the agent can now semantic_search
```

`embedding` may instead be `"embedding_model": "<alias>"` pointing at a model in `models`. If neither is set, coded guesses a default embedding model from your active provider. `grep` is still there for exact/regex matches.

Web search & fetch

`web_fetch` reads a URL (HTML stripped to text). `web_search` needs a backend — it auto-detects from whichever is set:

Backend	Config / env
Tavily	`TAVILY_API_KEY`
Brave	`BRAVE_API_KEY`
SerpAPI	`SERPAPI_API_KEY`
SearXNG (keyless)	`SEARXNG_URL` (your instance)

Or pin it in config: `"web_search": { "backend": "tavily", "api_key": "..." }`.

Image input (vision)

Attach images for a vision-capable model. Mark the model `"supports_vision": true`, then:

```
coded --image screenshot.png "what's wrong with this UI?"
# or in the REPL:
/image mockup.png build this layout as HTML
# or mention it, or paste it with Ctrl+V:
look at @mockup.png and build this layout as HTML
```

Checkpoints & undo

Every turn, changes made by `write`/`edit`/`move`/`delete` are snapshotted under `.coded/checkpoints/`. `/undo` restores the most recent turn (reverting edits and removing newly-created files); repeat `/undo` to step further back. `/checkpoints` lists them. Note: changes made by `bash` are **not** tracked.

`/rewind` goes back further than files: `/rewind` lists your requests, and `/rewind 3` removes request 3 and everything after it from the conversation *and* undoes the file changes those turns made, then puts request 3 back in the input so you can change it and send it again. `--keep-files` rewinds only the conversation.

Context and exporting

`/context` shows what fills the model's context window — system prompt, tool definitions, your messages, the model's replies and tool results — as estimated tokens and a share of the window, next to the count the endpoint reported for the last request. `/export [path]` saves the conversation as Markdown (default `coded-session-<time>.md` in the project), with tool calls and their results.

With `--print`, `--output-format stream-json` prints one JSON object per line while the agent works — `start`, then the agent's events (`assistant_delta`, `tool_call`, `tool_result`, `diff`, `todos`, `status`, …) and finally `result` with the answer, usage and cost — so a script or CI job can follow progress.

Git & GitHub

The `git` tool runs read-only subcommands (status/diff/log/blame/…) without prompting and gates mutations (commit/checkout/push/…). The `github` tool needs `GITHUB_TOKEN` and auto-detects the repo from the `origin` remote:

```
export GITHUB_TOKEN=ghp_...
coded "open a PR from this branch to main summarizing the changes"
```

Skills

Skills are Claude Code-style instruction packs: a directory with a `SKILL.md` (YAML frontmatter + Markdown body), loaded with **progressive disclosure**.

1. **At startup**, only each skill's `name` + `description` goes into the system prompt (cheap), so the model knows what exists and when to use it.
2. When a task matches, the model calls the `skill` tool (or you run `/skill <name>`), which loads the **full** `SKILL.md` body into the conversation.

3. The body can point to other files/scripts in the skill folder, which the agent reads with `read` or runs with `bash`.

Built-in role skills. coded ships with ready-made agent roles that are always available: **`planner`** (turn a task into an ordered plan), **`reviewer`** (review a diff for bugs/quality), **`qa`** (write and run tests), and **`security`** (audit for vulnerabilities). The agent invokes them automatically when a task matches, or you can run one directly:

```
/skill planner add rate limiting to the API
/skill reviewer            # reviews your current git diff
/planner add rate limiting # the same: every skill is also a slash command
```

They're written to use the `task` sub-agent tool for isolated work, so a review or investigation doesn't clutter the main conversation.

`/review` pipeline. The `/review [target]` command runs `reviewer`, then `qa`, then `security` as sequential, isolated sub-agents and aggregates their reports into one summary. With no target it reviews your current uncommitted changes (via the `git` tool):

```
/review                      # review the current git diff
/review coded/agent.py       # review a specific file
```

For CI, the same pipeline is a non-interactive subcommand that ends with a synthesized pass/fail verdict and sets the exit code:

```
coded review --yolo                      # review the diff; exit 1 if NEEDS_CHANGES
coded review src/ --output review.md     # review a target, save the report
coded review --fail-on never             # always exit 0 (report only)
coded review --no-verdict                # skip the verdict/gate step
```

`--yolo` lets the qa/security phases run tests and scanners via `bash` (otherwise those actions are denied in non-interactive mode and the review is read-only). Example GitHub Actions step:

```
- run: pip install -e . && coded review --yolo -o review.md
  env:
    OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
```

Discovery locations (a later location overrides an earlier one on name clash, so your own `planner` shadows the built-in):

```
<package>/builtin_skills/<name>/SKILL.md  # bundled roles (lowest priority)
~/.config/coded/skills/<name>/SKILL.md    # user-global
./.coded/skills/<name>/SKILL.md           # project-local (highest priority)
```

Try the bundled example:

```
mkdir -p .coded/skills
cp -r examples/skills/conventional-commit .coded/skills/
coded            # then: /skills
```

A minimal `SKILL.md`:

```
---
name: my-skill
description: What it does, and WHEN to use it (this drives auto-invocation).
allowed-tools: read, edit, bash
---

# My skill
Step-by-step instructions the agent follows once the skill is invoked…
```

See `examples/skills/` for a working example and a guide. Use `--no-skills` to disable discovery for a run.

MCP servers

Add servers under `mcp_servers` in config (install extras with `pip install -e '.[mcp]'`):

```
{
  "mcp_servers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."]
    }
  }
}
```

Their tools appear to the agent as `mcp__<server>__<tool>` and go through the same permission gate.

When a server offers resources (files, docs, records), the agent also gets `list_mcp_resources` and `read_mcp_resource`; without any, those two tools are not sent.

Values like `"${GITHUB_TOKEN}"` (or `"${VAR:-default}"`) in `command`, `args` and `env` are filled in from your environment when coded starts. A reference to a variable that is not set is reported by name at start (and used as empty).

Reuse the servers you set up for other agents. `coded import` looks for MCP servers configured for Claude Code (`~/.claude.json`, the project's `.mcp.json`), Claude Desktop, Cursor, VS Code (`.vscode/mcp.json`), Gemini CLI, Codex (`~/.codex/config.toml`) and Windsurf, shows each one's command verbatim, and asks which to copy into your coded config:

```
coded import --list                # what it found; changes nothing
coded import                       # pick which to copy (user config)
coded import --from claude,cursor  # only these agents
coded import --only github -y      # one server, no question
coded import --project             # into ./.coded/config.json
coded import mcp                   # only MCP servers (`commands`: only slash commands)
```

`coded import` also copies Claude Code's custom slash commands (see Custom slash commands).

A server name you already have is left alone. URL (http/sse) servers are listed as skipped: coded runs stdio servers only. Literal `env` values (usually keys) go only into the user config, which is then written with mode 600; the project config, which is shared, refuses them but takes `${VAR}` references.

Task list & plan mode

For multi-step work the agent keeps a visible task list (`todo_write`), shown as it changes and kept right above the input box — in the TUI always, in the REPL before each prompt while a task is unfinished (`/todos` to show it again). The REPL's input sits in a box like the TUI's, with the status line right under it. Each update shows progress — `Tasks 2/5 ▰▰▰▰▱▱▱▱▱▱ 40%`, the running item marked `← now` — and the status bar carries the same count and the current item while the agent works, so a long turn is never a silent one.

You can type while it works. The input never goes away: while the agent — or a command that runs it, like `/review`, `/init`, a skill or a custom command — is working, an input box stays at the bottom with the execution state above it (`◆ running tool bash · 1 follow-up waiting`, `? waiting for you`, `■ interrupting`), and what you type is routed by what it is:

- **A message** goes to the *running* agent: it is delivered at the next step (`↪ follow-up delivered: …`), or just before the agent would finish, so it can change course — "use port 8080", "also update the README", an answer to something it said. Several are delivered together, in order.
- **A read-only command** (`/btw`, `/dashboard`, `/tasks`, `/todos`, `/cost`, `/context`, `/help`) answers at once, alongside the work.
- **Any other command** runs right after the work finishes (`⏎ queued … · runs after this one`). Commands that don't run the agent loop (`/review`, `/redteam`, `/compact`, `/verify`) also keep the box; messages typed during those are sent when they finish.
- **Esc** (or Ctrl-C) stops the work. Nothing you typed is lost or run behind your back: it is put back into the input to send, edit or clear.

Typing `/` lists the matching commands (see *Command suggestions*). **↑/↓** walk your earlier messages (↑ first takes back a queued one), **Ctrl+R** finds the newest earlier message containing what you typed (again: older), **Ctrl+L** clears the screen. The permission prompt takes the next line you type, so the two never fight over the same keystrokes. The same routing works in the TUI (Esc stops a turn at once — a reply still streaming, a running shell command and its children — when no suggestion list is open); without a terminal (piped input, `--print`) it stays off. Between requests the status bar shows how the last one ended (`✓ completed 14s`, `✗ failed`, `■ interrupted`).

That input box is a full line editor:

Keys	What they do
←/→	One character
Ctrl+←/→, **Option/Alt+←/→** (or Alt+B/F)	One word
Home/End, **Ctrl+A/E**	Start/end of the line; **Ctrl+Home/End**: of the whole message
↑/↓	Between the lines of a multi-line message (↑ on the first line edits the newest queued message)
Shift + any move	Select; typing, Backspace or Delete replace the selection
Backspace / **Delete**, **Ctrl+D**	Delete before / after the cursor
Ctrl+W, **Alt+Backspace** / **Alt+D**	Delete the word before / after
Ctrl+U / **Ctrl+K**	Delete to the start / end of the line

Keys	What they do
`\` then Enter, **Alt+Enter** (Windows: **Ctrl+Enter**)	New line; Enter queues the whole message

On Windows, Shift-selection reads the Shift key as the arrow arrives (`GetAsyncKeyState`); Ctrl+J is Enter on POSIX terminals, so use `\`+Enter or Alt+Enter there.

Plan mode (`coded --plan`, or `/plan` / **Shift+Tab** in the REPL/TUI) makes the agent research with read-only tools only. When it has a plan it calls `exit_plan_mode`; you approve (implementation starts) or reject (it revises). Every risky action is refused while plan mode is on, including `git`/`github` writes and sub-agents' actions. The agent can also switch itself into plan mode (`enter_plan_mode`) before a large or risky change (only when someone is there to approve the plan; in `--print` and CI it is told to do the work directly), and ask you a question with a few choices when a decision is yours (`ask_user`): answer with a number or your own words, or press Enter to skip. Without anyone to answer (`--print`, ACP) it states its assumption and carries on.

With a vision model (`"supports_vision": true`) the agent can look at image files itself (`view_image`): screenshots, diagrams, mockups. The tool is not offered to models that cannot see.

Verification: don't claim success you haven't earned

A turn that changed something can be made to prove it before you see the answer:

```
{
  "verify": {"command": "pytest -q", "second_opinion": true, "max_rounds": 2},
  "model_roles": {"verify": "big"}
}
```

Two gates, both off until configured:

1. **The project's own checks.** `verify.command` runs after any turn that touched something (file tools, `bash`, `git` — not just checkpointed edits). A non-zero exit sends the turn back to the agent with the output, telling it to fix the cause and **never** to weaken, skip or delete a test — and to say so and stop if the failure predates its change.
2. **A second opinion.** With `model_roles.verify` set, a *different* model reads your request and the diff and answers `PASS` or `NEEDS_CHANGES`. One model marking its own homework is a known weak spot; a reviewer that didn't write the code is the cheapest correction available when you have more than one endpoint — and in coded they can be from different vendors. The agent is told the reviewer may be wrong, so it can push back in its answer instead of obeying.

Either complaint costs a round; after `max_rounds` the answer is delivered with a visible "unverified" note rather than a silent pass. The second opinion runs only when a verify model is *named* — it is never auto-assigned, so a small local model is not promoted to critic by accident — and an unparseable verdict or a reviewer outage releases the turn instead of blocking it. Sub-agents, plan mode and unchanged turns are never verified, so an unconfigured session sends exactly the requests it did before.

`/verify` runs the gate on demand, and `coded config setup` offers to configure it. The command runs through the same shell as the `bash` tool, so on Windows write it the way that shell expects — Git Bash treats a backslash as an escape, so `C:/tools/python.exe -m pytest` works where the backslashed path silently does not.

Hooks

Run your own commands on agent events. Each hook gets a JSON payload on stdin (`hook_event_name`, `cwd`, `tool_name`, `tool_input`, `tool_response`, `prompt`). Exit `2` to **block** (stderr goes back to the model); exit `0` to continue (stdout of `UserPromptSubmit`/`SessionStart` hooks is added as context). A `PreToolUse` hook may print `{"decision": "allow"}` to skip the prompt or `{"decision": "block", "reason": "..."}` to block.

```
{
  "hooks": {
    "PreToolUse":  [{"matcher": "bash", "command": "./scripts/guard-shell.sh"}],
    "PostToolUse": [{"matcher": "edit|multi_edit|write|apply_patch",
                     "command": "ruff format -q . && ruff check -q . >&2 || exit 2"}],
    "UserPromptSubmit": [{"command": "cat .coded/sprint-notes.md"}],
    "SessionStart": [{"command": "git status --short"}],
    "Stop": [{"command": "pytest -q -x >&2 || exit 2", "timeout": 300}]
  }
}
```

`matcher` is a regex on the tool name (omit for all). The Claude Code form `{"matcher": ..., "hooks": [{"type": "command", "command": ...}]}` also works. A `Stop` hook that exits 2 makes the agent keep working (at most 3 times per turn).

Sandbox

```
coded --sandbox                    # bash writes only inside the project; no network
coded --sandbox --sandbox-network  # …but allow network
```

or in config:

```
{ "sandbox": { "enabled": true, "network": false, "writable": ["~/.cache/pip"],
               "auto_allow": true, "fail_if_unavailable": false } }
```

On Linux this needs bubblewrap (`apt install bubblewrap`); macOS uses the built-in `sandbox-exec`. Inside the sandbox the filesystem is read-only except the working directory, `writable` paths and a private `/tmp`. Because the sandbox is the safety boundary, sandboxed commands run **without a permission prompt** (`auto_allow`). Deny rules, plan mode and the strict injection guard still apply. The model can ask to run one command unsandboxed (`dangerously_disable_sandbox`), which always needs your approval. If no backend is installed, bash falls back to normal prompting (or refuses, with `fail_if_unavailable`).

Windows

coded runs natively on Windows. Everything works except the OS sandbox: there is no Windows backend, so `--sandbox` falls back to the normal approval prompt for every command (set `"fail_if_unavailable": true` in the `sandbox` config to refuse to run instead). WSL gives you the sandbox along with Linux behaviour.

The `bash` tool needs a shell. coded picks one at startup:

1. `"shell"` in config.json, `$CODED_SHELL`, or `--shell`;

2. **Git Bash** (installed with Git for Windows) — recommended, because it understands the POSIX syntax models write;
3. PowerShell (`pwsh`, then `powershell`), driven with `-EncodedCommand` so quoting is never mangled;
4. `cmd.exe`.

```
{"shell": "pwsh"}
```

```
coded --shell "C:/Program Files/Git/bin/bash.exe"
```

The chosen shell is named in the system prompt, and when it isn't POSIX the model is told to write that shell's syntax (PowerShell needs `&` to run a quoted executable, `;` to chain, `$env:NAME` for variables). PowerShell's serialized CLIXML error output is turned back into plain text. `bash.exe` in `System32` is the WSL launcher, not a shell for the current directory, so it is skipped.

Other Windows specifics, all automatic:

- **Line endings are preserved.** Editing one line of an LF file leaves the rest of the file alone (no silent CRLF rewrite), and CRLF files stay CRLF.
- **Output is decoded as UTF-8** from commands, hooks, git and outside agents, and the console is switched to UTF-8 so the UI renders.
- **Stopping a command kills its children** (`taskkill /T`), for timeouts, `kill_shell` and shutdown.
- **Outside agent CLIs** installed as `.cmd`/`.bat` shims (claude, codex, …) are resolved to their real path.

Outside agent CLIs installed as `.cmd`/`.bat` shims are run with the prompt as an argument, which `cmd.exe` re-parses; coded refuses a prompt containing `" & | < > ^ %` for such an agent and tells you to set `"stdin": true` for it (or point `command` at the real `.exe`).

Background processes

`bash` with `run_in_background: true` starts dev servers, watchers or long builds without blocking. The agent polls new output with `bash_output` (with an optional regex `filter`) and stops them with `kill_shell`. `/shells` lists them; all are stopped when coded exits.

Sub-agents as tasks

Every sub-agent the agent starts with the `task` tool is a **task**. While one runs, the bottom of the REPL shows it with its model, running time, tool uses and latest steps:

```
● Task(Review API test suites)  t1 · qwen3-14b · 4 tool uses · 5s
  ⎿  Read(tests/api/test_auth.py)
     Search(dmFtc2)
     Read(tests/api/test_users.py)
```

and when it ends, `⎿ Done (12 tool uses · 1m 30s)` is printed under its call. **Ctrl+O** shows all of its steps; **Ctrl+B** moves it to the background, so the turn ends and you can keep chatting. The agent can also start one there itself (`task` with `run_in_background: true`), check on it (`task_output`) or stop it (`task_stop`). When a background task finishes, its report is handed to the agent: at once if you are

idle at the prompt, with your next message if you are typing, or at the agent's next step if a turn is running. The status bar shows `● N running tasks` meanwhile, and `/tasks` (**Ctrl+T**) lists them all.

A background task cannot ask for approval: an action that would need it is refused (allow rules, `--yolo` and accept-edits mode still apply), and the task says so in its report. In `coded -p`, the TUI and ACP, where nothing can wake the agent later, a turn waits for its background tasks before it ends.

Multiple endpoints, models and API keys

Every model entry has its own endpoint and key (`base_url`, `api_key`, or `api_key_env`), so you can mix providers, self-hosted servers and accounts. On top of that:

```
{
  "models": {
    "main":   {"provider": "openai", "model": "gpt-4.1", "api_key_env": "OPENAI_KEY_TEAM",
               "fallback": ["main-or", "local"], "max_retries": 4},
    "main-or": {"provider": "openrouter", "model": "openai/gpt-4.1"},
    "cheap":  {"provider": "groq", "model": "llama-3.3-70b-versatile"},
    "local":  {"base_url": "http://localhost:11434/v1", "model": "qwen2.5-coder:14b"},
    "thinker": {"provider": "openai", "model": "o3-mini", "reasoning_effort": "high"}
  },
  "model_roles": {"subagent": "cheap", "compact": "cheap", "review": "thinker", "verdict":
"main"}
}
```

- **Retries**: 429/5xx/connection errors are retried with jittered exponential backoff (honoring `Retry-After`), `max_retries` / `retry_base_delay` per model. A stream that breaks before any text was shown is retried the same way.
- **Timeout**: `"timeout": 900` (seconds) per model: how long to wait for the endpoint to connect/respond and, while streaming, between chunks. Unset = 10 minutes. Raise it for slow, very large models.
- **Fallback**: when a model still fails (or its key is rejected), coded moves on to the next alias in `fallback`, e.g. the same model through another provider or key, then a local model.
- **Cooldown**: a model that failed over is skipped for `fallback_cooldown` seconds (default 120; `0` = try it first every time), so an outage doesn't cost a full retry cycle on every step. It is still tried last if every other model in the chain fails.
- **Smaller fallbacks**: while a fallback with a smaller `context_window` is in use, coded sizes tool output and compaction for *that* model and shrinks/summarizes the existing conversation to fit. It returns to the main model's sizes when the main model is back. Detail is lost in the summary, so prefer fallbacks with a similar window.
- **Typos**: an alias in `fallback`, `model_roles` or `default_model` that isn't a configured model is reported at startup and by `coded models`.
- **Roles**: `model_roles` picks the model for sub-agents (`subagent`), context compaction (`compact`), the review phases (`review`), the CI pass/fail call (`verdict`) and plan mode (`plan`). With `"plan": "large"`, plan mode runs on `large` and coded returns to your main model to implement the approved plan. The `task` tool also accepts a `model` alias per sub-agent.
- **Health check**: `coded models` lists endpoints, fallbacks and roles; `coded models --check [alias ...]` sends a tiny request to each and reports latency or the error.
- **Reasoning**: `reasoning_effort` is sent to reasoning models, `extra_body` passes provider-specific fields through as-is, and `--show-reasoning` / `"show_reasoning": true` displays streamed

thinking (`reasoning_content`). An optional parameter that an endpoint rejects is dropped automatically and remembered.

Routing by difficulty

Off unless configured. When it is on, coded reads each request and picks the model for that request — one decision per request, never mid-task:

```
{
  "routing": {
    "enabled": true,
    "simple": "small",
    "medium": "medium",
    "hard": "large",
    "sticky": true,
    "rules": [{"match": "(?i)^deploy\\b", "level": "hard"}]
  }
}
```

- The classifier is a **heuristic over the text of your request** — keywords (refactor, debug, race condition, security, "why does…"), length, and your own `rules` (regexes, tried first). It costs nothing and makes no model call.
- A request it can't read — "fix it", "yes", "now the tests" — **keeps the model you are already on**, because a short follow-up continues the last task.
- `"sticky": true` (the default) means a session never drops back down after a hard request. Set it to `false` to let every request stand on its own.
- coded **never routes down into a model your conversation no longer fits** (that would summarize your history to save pennies), and skips a target that can't run tools, or can't see images when you attached some.
- `-m` or `/model` **pins** your choice and turns routing off; `/route on` puts it back. `/route` alone shows the state, the targets, and the current model.
- Each decision is announced in the session ("Routing: hard request → model 'large'"), so you can see what it did.

It is a heuristic, not a measurement: treat the levels as a cost control, and pin the model yourself when a request matters more than it looks.

Model pools

Several interchangeable models behind one name:

```
{
  "pools": {"big": ["large-a", "large-b"]},
  "default_model": "big",
  "model_roles": {"subagent": "big"}
}
```

A pool name works anywhere a model alias does — `coded -m big`, `/model big`, `default_model`, `model_roles`, and routing targets.

- A member is picked when the pool is resolved: at session start, per sub-agent, per routed request. The agent then **stays on that member** for the session, so the endpoint keeps its

prompt cache.
- The other members become its **fallbacks**, ahead of the member's own `fallback` list.
- A member that fails is **skipped for everyone** until its cooldown passes (`fallback_cooldown`); endpoint health is shared across the process, including sub-agents.
- Give members the same `context_window` — `coded models` warns if they differ, because each request is sized for the member that was picked.
- `coded models` lists pools; `coded models --check big` checks every member.

Large and small models

coded adapts to each model's `context_window`. Set it accurately: it's the main knob, and the default is 128k.

- **Limits scale with the window.** These all grow or shrink in proportion (×window/128k, clamped to ⅛×–8×, with floors for small models), so a 300k model reads ~4,700 lines per call and keeps ~70k characters of command output, while a 16k model gets 250 lines and 4k characters:
- lines per `read`
- bash, git and outside-agent output kept
- `web_fetch` page size
- grep matches
- instruction-file budgets
- messages kept verbatim when compacting

At 128k the limits are the same as before. - **Safety caps.** No single tool result may exceed ~30% of the window, and all results of one step together ~60%. At 128k that's loose enough for a full 2,000-line read or a normal batch of parallel reads. `read` stops at whole lines and tells the model which `offset` to continue from; other output is cut with a note asking for a narrower request. If an endpoint still rejects a request as too long, coded halves that step's tool results, compacts and retries once. It recognizes "too long" by an HTTP 400 or 413 whose message contains phrases such as "context length", "maximum context", "context window", "too many tokens", "prompt is too long" or "request too large". If your server words it differently, that retry won't fire. - **Tune per model** with `limits`. Unknown keys are reported at startup:

```
json "limits": {"read_lines": 10000, "shell_output_chars": 200000, "tool_result_chars": 150000}
```

Keys: `read_lines`, `shell_output_chars`, `git_output_chars`, `web_fetch_chars`, `grep_matches`, `external_output_chars`, `instructions_file_chars`, `instructions_total_chars`, `nested_instructions_chars`, `compact_keep_recent`, `tool_result_chars`, `step_result_chars`, `repo_overview_chars`. - **Prompt sets** (`prompt_profile`):

Value	Meaning
`standard`	The default set
`compact`	Terse, for small or local models; still includes the safety rules
`full`	More thorough `coder`, `planner`, `reviewer` and `debugger` instructions, for large models
`auto` (default)	Picks by context window: ≤32k → compact, ≥200k → full

coded can't see parameter counts, so set `prompt_profile` explicitly for a 700B model with a modest window, or a small model with a huge one. - **Per-model prompt files.** `prompts_dir` points at a

folder of that model's own prompt files, which win over everything else. A relative path is resolved against the project directory. - **Longer runs.** `max_turns` (default 100) is the number of steps one request may take; set it per model or globally. When a request reaches it while still making progress, the limit grows by `max_turns` again, up to five times `max_turns`, with a note each time. It stops early when the agent looks stuck: the same tool call 4 times in its last 30, or 10 failing calls in a row. Checking on something still running (`bash_output`, `task_output`) is waiting, not a loop, and those tools take `wait` (seconds) to block until there is new output or the job is done, instead of using a step per check. After a stop, type `continue` to pick up where it left off. `"auto_extend_turns": false` makes `max_turns` a hard cap, and so does an explicit `--max-turns N`. - **Repository overview** (`repo_overview`: `auto`, `true` or `false`; auto means on for windows ≥200k). Large-context models get a map of the repository in the system prompt: file paths (or directories with counts for big repos), key manifests and recent commit subjects, but no file contents. Sub-agents don't get it. - **Switching models** with `/model` rebuilds the system prompt for the new model's prompt set and limits. Each sub-agent uses its own model's limits and prompt set. - **Checking the result:** - `coded models` shows each model's context size and prompt set. - `coded prompts list -m big` shows which file each prompt comes from for that model. - `coded prompts show -m big` prints the exact system prompt it gets.

```
{
  "models": {
    "big":   {"base_url": "https://llm.internal/v1", "model": "my-700b", "api_key_env":
"INTERNAL_KEY",
              "context_window": 300000, "max_tokens": 32000, "max_turns": 400,
              "prompt_profile": "full", "limits": {"read_lines": 8000}},
    "local": {"provider": "ollama", "model": "qwen2.5-coder:7b", "context_window": 32768,
              "prompt_profile": "compact"}
  },
  "default_model": "big",
  "model_roles": {"subagent": "local", "compact": "local", "review": "big", "verdict": "big"}
}
```

Outside agents

If other agent CLIs are installed, the model can hand work to them with the `external_agent` tool. It's useful for a second opinion, another model's strengths, or parallel work. Presets (used when the binary is on `PATH`): `claude` (Claude Code), `codex`, `gemini`, `aider`, `opencode`, `cursor` (`cursor-agent`), `goose`. Add your own or override presets:

```
{
  "external_agents": {
    "claude": {"command": ["claude", "-p", "{prompt}", "--model", "opus"]},
    "in-house": {"command": ["./bin/agent", "--stdin"], "stdin": true,
                 "description": "Our deployment agent", "timeout": 3600,
                 "env": {"AGENT_MODE": "ci"}},
    "aider": {"enabled": false}
  }
}
```

Each call needs permission (it runs another program), and can use `isolation: "worktree"` so the outside agent's edits land on their own branch. `/agents` shows what's available; `"detect_external_agents": false` turns auto-detection off.

`coded agents` shows every supported agent: whether it is installed and ready, whether it is open source, and — for the ones you don't have — the command to install it on your platform (and a missing prerequisite such as Node.js). coded prints these commands for you to run; it never installs anything itself.

```
coded agents                # status + install commands for the missing ones
coded agents --check        # start each installed agent with --version to prove it runs
coded agents aider goose    # just these
coded agents --json         # for scripts
```

Worktree isolation

`task` and `external_agent` accept `isolation: "worktree"`. The work happens in a fresh git worktree on a new `coded/<label>-<id>` branch under `~/.cache/coded/worktrees/`, so it can't collide with your working tree or with other agents. Several isolated tasks requested together run in parallel. If nothing changed, the worktree and branch are removed. Otherwise the result says where the changes are, for you to review and merge.

Prompts

coded's built-in prompts are plain Markdown files in `coded/builtin_prompts/`:

File	What it covers	Used by
`system.md`	Identity, tone, output style	main agent
`coder.md`	How to do the work: read first, small changes, verify, report	main agent, sub-agents
`tools.md`	Tool-calling rules: dedicated tools over bash, parallel calls, which edit tool, background jobs, delegation, handling denials	main agent, sub-agents
`security.md`	Untrusted content, secrets, destructive actions, sandbox, refusals	main agent, sub-agents
`skills.md`	How to use skills (heads the skill list)	main agent, when skills exist
`planner.md`	Planning method	`planner` skill; sent once when plan mode starts
`reviewer.md`	Review method	`reviewer` skill, `/review`, `coded review`
`debugger.md`	Root-cause debugging method	`debugger` skill

Override any file by putting your own version in `~/.config/coded/prompts/<name>.md` (for you) or `./.coded/prompts/<name>.md` (for one project; it wins over the user one). An override replaces the whole file; an **empty** file removes that section. Files are used verbatim, so braces and code samples are safe.

```
coded prompts list                    # each prompt, where it comes from, size
coded prompts show                    # the assembled main system prompt
coded prompts show subagent           # …or a sub-agent's, or one file: show reviewer
coded prompts init reviewer           # copy the built-in to ~/.config/coded/prompts/ to edit
coded prompts init all --project      # copy all of them into ./.coded/prompts/
```

Overriding `reviewer.md` also changes `/review` and `coded review`, since the `reviewer` skill reads its instructions from that file (same for `planner` and `debugger`). Smaller local models work best with short prompts, so trimming these files is a reasonable thing to do for them.

Installing skills and plugins

Skills and plugins can be installed from a local folder, a `.zip` file (a path, for example a file you uploaded, or a URL), any git URL, or GitHub shorthand `owner/repo[/sub/path][@ref]`.

```
coded skill install ./my-skill                      # a folder with SKILL.md
coded skill install ~/Downloads/skills.zip --all    # several skills in one archive
coded skill install acme/agent-skills/skills/lint@v1.2
coded skill list | update [name…] | remove <name> | info <name>

# From skills.sh: paste the page link, or the repository it points to
coded skill install https://skills.sh/vercel-labs/agent-skills/vercel-react-best-practices
coded skill install anthropics/skills --all               # a whole collection
coded skill install anthropics/skills pdf docx            # just some of it
coded skill install https://github.com/anthropics/skills/tree/main/skills/pdf

coded plugin install examples/plugins/code-quality
coded plugin install https://github.com/acme/coded-plugin.git --ref main
coded plugin list | info <name> | update [name…] | enable <name> | disable <name> | remove
<name>

coded plugin marketplace add acme/plugin-marketplace   # or a folder / zip / git URL
coded plugin search [query]
coded plugin install code-quality@coded-examples
coded plugin marketplace list | update | remove <name>
```

- **Skill collections (skills.sh and GitHub):** a skills.sh page is a link to a GitHub repository, so either link works; a page for one skill installs just that skill. Skills are found at the root, under `skills/*/`, `*/`, or nested deeper (`plugins/<p>/skills/<name>/`, `.claude/skills/<name>/`). Multi-line YAML descriptions and `metadata:` blocks are read correctly. A symlink elsewhere in the repository (such as `CLAUDE.md → AGENTS.md`) is dropped and ignored; one inside a skill you are installing is still refused. With many skills installed, the list in the system prompt is shortened to fit (`limits.skills_chars`): descriptions are cut, and past that the rest are listed by name — every skill can still be loaded by name.
- **What a plugin can contain:** skills, slash commands (shown as `/<plugin>:<command>`), hooks, MCP servers and outside agents. See `examples/plugins/` for the layout and an example. Claude Code plugins (`.claude-plugin/plugin.json`) install as-is:
- `${CLAUDE_PLUGIN_ROOT}` is expanded.
- Tool names in hook matchers (`Bash`, `Edit|Write`) are mapped to coded's.
- Parts coded can't use (sub-agent definitions, HTTP MCP servers, hook events such as `SessionEnd`) are listed as warnings.
- **Review before trusting:** before installing or updating, coded prints everything the plugin adds:

- every hook command and MCP server command, verbatim
- any scripts and executables it contains
- every warning

It then asks for confirmation. Without a terminal it refuses unless you pass `--yes`. Hooks run without a permission prompt, so installing a plugin lets it run code on your machine. - **Scope:** by default things install for your user (`~/.config/coded/`). `--project` installs into `./.coded/` instead. That folder is gitignored, so project installs aren't committed. - **Safe updates:** sources are fetched into a temporary folder and checked before anything is replaced. A failed update keeps the previous version. Zip archives with `..` paths, symlinks or oversized contents are rejected. - **Precedence and disabling:** - Skill names resolve built-in < plugin < user < project. - Plugin hooks are added after your own hooks; they never replace them. - On a name clash, your config's MCP servers and outside agents win. - `disable` turns off everything a plugin contributes without uninstalling it. - Changes take effect the next time coded starts. Only you can install, from the command line; the agent has no tool for installing skills or plugins.

Custom slash commands

Save reusable prompts as Markdown in `./.coded/commands/<name>.md` (project) or `~/.config/coded/commands/<name>.md` (user). Subfolders namespace them (`commands/git/pr.md` → `/git:pr`).

```
---
description: Fix a GitHub issue end to end
argument-hint: <issue-number>
model: smart            # optional: run this command on another model
---
Read issue #$1 with the github tool, reproduce it, fix it, add a test,
and summarize the change. Extra notes: $ARGUMENTS
```

Run with `/fix-issue 123`. `$ARGUMENTS` is the full argument string and `$1`…`$9` are individual arguments.

Already have Claude Code commands? They use the same format. `coded import commands` copies `~/.claude/commands` into `~/.config/coded/commands` and the project's `.claude/commands` into `.coded/commands` (`--project` puts them all in the project), shows each one first, and never overwrites a command you have. A `model:` that is not one of your coded models is dropped so the command still runs; Claude's `!`command`` lines are not run in advance (the summary points them out), and a name coded already uses (`/review`, `/init`, …) is skipped. Restart coded to use them.

Tracing & observability

`coded --trace trace.jsonl` (or `"telemetry": {"jsonl": "..."}` or `$CODED_TRACE_FILE`) writes one JSON span per line for each turn, LLM call (model, endpoint that answered, tokens, finish reason, duration) and tool call, linked by `trace_id`/`parent_id`, sub-agents included. With `"otel": true` and `pip install opentelemetry-sdk opentelemetry-exporter-otlp`, spans are also exported over OTLP (configure with the standard `OTEL_EXPORTER_OTLP_*` env vars).

Prompt-injection guard

Output from `web_fetch`, `web_search`, `github` and MCP tools is wrapped in `<untrusted-content>` and the model is told to treat it as data. Once such content is in the current turn, `injection_guard` decides what happens to risky actions:

- `"warn"` (default): permission prompts are flagged "⚠ untrusted content is in context".
- `"strict"`: auto-approval (`--yolo`, allow rules, "always") is suspended, so every risky action needs your explicit OK (and is denied when nobody can answer).
- `"off"`: no fencing or flagging.

Use coded from editors and other agents

- **Agent Client Protocol**: `coded acp` serves ACP over stdio, so ACP-capable editors (Zed, JetBrains, Neovim plugins, …) can use coded as their agent: streamed replies and thinking, tool-call cards, the task list as a plan, permission requests in the editor, `default`/`plan` modes, and cancel. Zed example (`settings.json`):

```
json { "agent_servers": { "coded": { "command": "coded", "args": ["acp", "-m", "main"] } } }
```

- **MCP server**: `coded mcp-serve` (needs `pip install 'coded[mcp]'`) exposes `coded_task(prompt, cwd?, model?)` and `coded_review(target?, cwd?)` to any MCP client, e.g. `claude mcp add coded -- coded mcp-serve`. Risky actions are denied unless started with `--yolo` (nobody is there to approve them) or allowed by permission rules.

Making it faster

Most of the wait is the model. The parts coded controls:

- **Measure first.** `coded models --speed` times every configured model (or the ones you name) on the request a real turn sends — coded's system prompt, the tool definitions and a question — and shows time to the first token, total time, output tokens per second and prompt size. `--runs 3` takes the median; `--minimal` sends only the question, to see the bare model.

```
| model | first token | total | speed | prompt | output | | big-model | 1.22s | 4.10s | 22 tok/s | 7,900 | 64 | | small-fast | 0.16s | 0.90s | 86 tok/s | 7,900 | 64 |
```

- **Don't send tools you never use.** Every tool's definition goes with every request (about 4k tokens for all of them). Names or patterns, globally or per model:

```
json { "disabled_tools": ["github", "web_*", "semantic_search", "mcp__slack__*"], "models": { "local-7b": { "disabled_tools": ["task", "repo_map", "memory"] } } }
```

A disabled tool is not offered, and a call to one is refused with a reason. - **Compact prompt profile** (`"prompt_profile": "compact"`, automatic at 32k context or less) now also sends shorter tool descriptions — about 20% fewer tool tokens, with the rules that prevent mistakes kept. - **Faster failover:** `"max_retries": 1` on a flaky model, plus a `fallback`. - **Route by difficulty** to a quick model for simple requests (see routing), using the `--speed` numbers to choose. - **Lower reasoning effort** (`--reasoning-effort low`) on reasoning models. - **Server-side prefix caching** (vLLM `--enable-prefix-caching`; llama.cpp caches by default): coded's system prompt and tools are identical between requests, so a caching server only reads them once. - **Startup:** the model SDK now loads

in the background, so `coded --version`, `coded agents`, `coded skill …` and the first prompt appear in ~0.2 s instead of ~0.7 s.

Checking your setup

`coded doctor` checks everything coded depends on and prints one line per area: Python, the required packages and which extras are installed, the config files, every model and whether its API key is found, the shell, the sandbox, git, the terminal, and helpers like the clipboard reader. `coded doctor --network` also sends each model's endpoint the staged probe from `coded models --check`. It exits with 1 when something is broken, so it works in scripts.

When coded runs from a git clone, it checks once a day in the background whether the clone's remote has moved on, and says so at the next start (`git -C <clone> pull` updates it). Turn that off with `"update_check": false`.

Reviewing pull requests in GitHub Actions

`examples/github-actions/coded-review.yml` runs `coded review` on every pull request, posts the reviewer → qa → security report as a PR comment (updated on each push) and fails the check when the verdict is NEEDS_CHANGES. Copy it to `.github/workflows/`, and add a `CODED_API_KEY` secret plus `CODED_BASE_URL` and `CODED_MODEL` variables for any OpenAI-compatible endpoint.

When a model errors

If a turn ends with something like `InternalServerError: Error code: 500`, that status came from the endpoint. coded names the model and its base URL and retries first (`max_retries`, with backoff and `Retry-After`), then any `fallback` models. To find out what the endpoint will actually accept:

```
coded models --check glm-5-3-flash
```

The check is a staged probe, not one ping: a gateway that answers "hi" can still fail on streaming, on `stream_options` or on tool definitions — and some answer a parameter they don't implement with a 500 rather than a 400, which looks exactly like an outage. It stops at the first shape refused and says what to change:

```
✗ glm-5-3-flash         74 ms
      ✓ minimal request
      ✓ streaming
      ✓ stream_options
      ✗ tools: InternalServerError: Error code: 500 - An unexpected error occurred
      → this endpoint cannot take tool definitions. Set "supports_tools": false
        for this model — coded will then run it without tools.
```

The last stage sends the shape a real turn uses (system prompt + streaming + tools), because each part can pass alone and fail together.

During a turn coded does a narrower version of this by itself: once the retries for a 5xx are exhausted it tries once more with the optional parameters (`stream_options`, `reasoning_effort`, `parallel_tool_calls`) removed, says so, and remembers them as unsupported if that works. It never drops `tools` automatically — an agent that silently loses its tools is worse than an error.

If a model doesn't work, coded tries another. After its own retries, and after any `fallback` models you declared, it sweeps the rest of your configured models rather than ending the turn — closest first (same endpoint, then the widest context window), one attempt each, and it says which one answered. A model that can't serve the request is skipped: no vision model for an image, no tool-less model for a turn that needs tools, and nothing whose context window would force the conversation to be summarized. The error you finally see names the model *you* chose and lists what else was tried.

This can send a request to a different provider than the one you picked (local → cloud, say). To stay strictly on the models you declared:

```
{"auto_fallback": false}
```

To control the order instead, declare the fallback yourself:

```
{"models": {"glm-5-3-flash": {"fallback": ["gpt-oss-120b"]}}}
```

Development

```
pip install -e '.[dev]'
pytest
```

The test suite spins up a fake OpenAI-compatible server to exercise the full agent loop (tool calls, streaming, and the permission gate) without network access or API keys.

See `CLAUDE.md` for an architecture overview aimed at AI assistants and contributors.

License

MIT

2. Writing Skills

How to write and install skill packs. — from `examples/skills/README.md`

Skills are Claude Code-style, progressively disclosed instruction packs. Each skill is a directory with a `SKILL.md` file (YAML frontmatter + Markdown body).

Install a skill

The easy way: `coded skill install` takes a folder, a `.zip`, a git URL or GitHub `owner/repo[/path][@ref]`, shows what's inside, and records the source so `coded skill update` works:

```
coded skill install examples/skills/conventional-commit
coded skill install examples/skills/conventional-commit --project   # ./.coded/skills
```

Or copy a skill directory into either location by hand (project skills override user skills of the same name):

```
# Project-local (this repo/working dir):
mkdir -p .coded/skills
cp -r examples/skills/conventional-commit .coded/skills/

# Or user-global:
mkdir -p ~/.config/coded/skills
cp -r examples/skills/conventional-commit ~/.config/coded/skills/
```

coded discovers them on startup. Confirm with `/skills` in the REPL, or `coded --print "..."`.

How they load (progressive disclosure)

1. **Startup** — only each skill's `name` + `description` is added to the system prompt, so the model knows what exists and when to use it.
2. **On demand** — the model calls the `skill` tool (or you run `/skill <name>`), which loads the full `SKILL.md` body into the conversation.
3. **As needed** — the body can reference other files/scripts in the skill directory; the agent reads them with `read` or runs them with `bash`.

Write your own

```
.coded/skills/my-skill/
├── SKILL.md          # required: frontmatter + instructions
├── reference.md      # optional: extra docs the body points to
└── script.py         # optional: code the agent runs with bash
```

`SKILL.md` frontmatter:

```
---
name: my-skill
description: What it does, and WHEN to use it (drives auto-invocation).
allowed-tools: read, edit, bash   # optional, informational
---
```

Write the `description` for triggering — say both what the skill does and the situations it applies to.

3. Writing Plugins

The plugin layout: skills, commands, hooks and MCP servers in one package. — from `examples/plugins/README.md`

A plugin bundles skills, slash commands, hooks, MCP servers and outside agents into one installable directory.

```
# Install straight from this folder:
coded plugin install examples/plugins/code-quality

# Or add the examples folder as a marketplace and install by name:
coded plugin marketplace add examples
coded plugin search
coded plugin install code-quality@coded-examples
```

Layout

```
my-plugin/
├── .coded-plugin/plugin.json   # manifest (or .claude-plugin/plugin.json)
├── skills/<name>/SKILL.md      # skills
├── commands/<name>.md          # slash commands → /my-plugin:<name>
├── hooks/hooks.json            # {"hooks": {"PostToolUse": [...]}}
└── .mcp.json                   # {"mcpServers": {"name": {"command": ..., "args": [...]}}}
```

Manifest fields: `name` (required, lowercase), `version`, `description`, `author`, `license`, and optionally `hooks` / `mcpServers` (inline objects or paths), `skills` / `commands` (extra directories), and `external_agents` (same shape as the config key).

Use `${CODED_PLUGIN_ROOT}` (or `${CLAUDE_PLUGIN_ROOT}`) in hook and MCP commands to refer to the installed plugin directory. Claude Code plugins (`.claude-plugin/plugin.json`) install as-is: tool names in their hook matchers (`Bash`, `Edit|Write`) are mapped to coded's, and unsupported parts (sub-agent definitions, non-stdio MCP servers, unknown hook events) are reported at install time.

A marketplace is any directory or repo with `.claude-plugin/marketplace.json` (or `.coded-plugin/marketplace.json`) listing `plugins`, each with a `name` and a `source`: a relative path (`./plugins/x`), `{"source": "github", "repo": "o/r"}`, or `{"source": "url", "url": "https://….git"}`.

4. Example Configuration

The annotated sample configuration (config.example.json). — from `config.example.json`

```
{
  "default_model": "gpt-4o",

  "models": {
    "gpt-4o": {
      "provider": "openai",
      "model": "gpt-4o",
      "context_window": 128000,
      "max_tokens": 16384,
      "temperature": 0.2,
      "input_cost": 2.5,
      "output_cost": 10.0,
      "max_retries": 4,
      "fallback": ["gpt-4o-openrouter", "claude-sonnet"]
    },
    "gpt-4o-openrouter": {
      "provider": "openrouter",
      "model": "openai/gpt-4o",
      "context_window": 128000,
      "max_tokens": 16384
    },
    "gpt-4o-mini": {
      "provider": "openai",
      "model": "gpt-4o-mini",
      "context_window": 128000,
      "max_tokens": 16384,
      "input_cost": 0.15,
      "output_cost": 0.6
    },
    "gpt-4.1": {
      "provider": "openai",
      "model": "gpt-4.1",
      "context_window": 1000000,
      "max_tokens": 32768,
      "input_cost": 2.0,
      "output_cost": 8.0
    },
    "o3-mini": {
      "provider": "openai",
      "model": "o3-mini",
      "context_window": 200000,
      "max_tokens": 100000,
      "reasoning_effort": "medium",
      "input_cost": 1.1,
      "output_cost": 4.4
    },

    "claude-sonnet": {
      "provider": "anthropic",
      "model": "claude-3-7-sonnet-latest",
      "context_window": 200000,
      "max_tokens": 8192,
      "temperature": 0.2,
      "input_cost": 3.0,
      "output_cost": 15.0
    },
    "claude-haiku": {
      "provider": "anthropic",
      "model": "claude-3-5-haiku-latest",
      "context_window": 200000,
      "max_tokens": 8192,
```

```
      "input_cost": 0.8,
      "output_cost": 4.0
    },

    "gemini-flash": {
      "provider": "google",
      "model": "gemini-2.0-flash",
      "context_window": 1000000,
      "max_tokens": 8192,
      "input_cost": 0.1,
      "output_cost": 0.4
    },
    "gemini-pro": {
      "provider": "google",
      "model": "gemini-1.5-pro",
      "context_window": 2000000,
      "max_tokens": 8192,
      "input_cost": 1.25,
      "output_cost": 5.0
    },

    "deepseek-chat": {
      "provider": "deepseek",
      "model": "deepseek-chat",
      "context_window": 64000,
      "max_tokens": 8192,
      "temperature": 0.2,
      "input_cost": 0.27,
      "output_cost": 1.1
    },
    "deepseek-reasoner": {
      "provider": "deepseek",
      "model": "deepseek-reasoner",
      "context_window": 64000,
      "max_tokens": 8192,
      "input_cost": 0.55,
      "output_cost": 2.19
    },

    "grok": {
      "provider": "xai",
      "model": "grok-2-latest",
      "context_window": 131072,
      "max_tokens": 8192,
      "input_cost": 2.0,
      "output_cost": 10.0
    },

    "mistral-large": {
      "provider": "mistral",
      "model": "mistral-large-latest",
      "context_window": 128000,
      "max_tokens": 8192,
      "input_cost": 2.0,
      "output_cost": 6.0
    },
    "codestral": {
      "provider": "mistral",
      "model": "codestral-latest",
      "context_window": 256000,
      "max_tokens": 8192,
      "temperature": 0.1,
```

```
    "input_cost": 0.3,
    "output_cost": 0.9
  },

  "llama-groq": {
    "provider": "groq",
    "model": "llama-3.3-70b-versatile",
    "context_window": 128000,
    "max_tokens": 32768,
    "input_cost": 0.59,
    "output_cost": 0.79
  },
  "llama-groq-fast": {
    "provider": "groq",
    "model": "llama-3.1-8b-instant",
    "context_window": 128000,
    "max_tokens": 8192,
    "input_cost": 0.05,
    "output_cost": 0.08
  },

  "llama-cerebras": {
    "provider": "cerebras",
    "model": "llama-3.3-70b",
    "context_window": 128000,
    "max_tokens": 8192,
    "input_cost": 0.85,
    "output_cost": 1.2
  },

  "qwen-coder-together": {
    "provider": "together",
    "model": "Qwen/Qwen2.5-Coder-32B-Instruct",
    "context_window": 32768,
    "max_tokens": 8192,
    "temperature": 0.1,
    "input_cost": 0.8,
    "output_cost": 0.8
  },

  "llama-fireworks": {
    "provider": "fireworks",
    "model": "accounts/fireworks/models/llama-v3p3-70b-instruct",
    "context_window": 128000,
    "max_tokens": 8192,
    "input_cost": 0.9,
    "output_cost": 0.9
  },

  "openrouter-sonnet": {
    "provider": "openrouter",
    "model": "anthropic/claude-3.7-sonnet",
    "context_window": 200000,
    "max_tokens": 8192,
    "input_cost": 3.0,
    "output_cost": 15.0,
    "extra_headers": {
      "HTTP-Referer": "https://github.com/krishnavams/coded",
      "X-Title": "coded"
    }
  },
```

```
    "sonar": {
      "provider": "perplexity",
      "model": "sonar-pro",
      "context_window": 200000,
      "max_tokens": 8192,
      "supports_tools": false,
      "input_cost": 3.0,
      "output_cost": 15.0
    },

    "kimi": {
      "provider": "moonshot",
      "model": "kimi-k2-0711-preview",
      "context_window": 128000,
      "max_tokens": 8192,
      "input_cost": 0.6,
      "output_cost": 2.5
    },

    "qwen-local": {
      "provider": "ollama",
      "model": "qwen2.5-coder:7b",
      "context_window": 32768,
      "max_tokens": 8192,
      "temperature": 0.2
    },
    "llama-local": {
      "provider": "ollama",
      "model": "llama3.1:8b",
      "context_window": 128000,
      "max_tokens": 8192
    },
    "lmstudio-local": {
      "provider": "lmstudio",
      "model": "your-loaded-model-id",
      "context_window": 32768,
      "max_tokens": 8192
    },
    "vllm-local": {
      "provider": "vllm",
      "model": "meta-llama/Llama-3.1-8B-Instruct",
      "context_window": 128000,
      "max_tokens": 8192
    },

    "custom": {
      "provider": "openai-compatible",
      "model": "my-model-id",
      "base_url": "http://localhost:9000/v1",
      "api_key_env": "MY_API_KEY",
      "context_window": 32768,
      "max_tokens": 4096,
      "supports_tools": true
    },
    "corp-internal": {
      "provider": "openai-compatible",
      "model": "internal-llm",
      "base_url": "https://llm.internal/v1",
      "api_key_env": "INTERNAL_API_KEY",
      "ca_bundle": "/etc/ssl/corp-ca.pem"
    },
    "big-700b": {
```

```
      "provider": "openai-compatible",
      "base_url": "https://llm.internal.example.com/v1",
      "api_key_env": "INTERNAL_API_KEY",
      "model": "your-700b-model",
      "context_window": 300000,
      "timeout": 900,
      "fallback_cooldown": 120,
      "max_tokens": 32000,
      "max_turns": 400,
      "prompt_profile": "full",
      "repo_overview": true,
      "limits": {"read_lines": 8000, "shell_output_chars": 120000},
      "fallback": ["gpt-4.1"]
    },
    "small-local": {
      "provider": "ollama",
      "model": "qwen2.5-coder:7b",
      "context_window": 32768,
      "prompt_profile": "compact",
      "prompts_dir": ".coded/prompts-small"
    },
    "self-signed": {
      "provider": "openai-compatible",
      "model": "local-llm",
      "base_url": "https://192.168.1.5/v1",
      "verify_ssl": false
    }
  },

  "embedding": {
    "provider": "openai",
    "model": "text-embedding-3-small"
  },

  "web_search": {
    "backend": "tavily",
    "api_key_env": "TAVILY_API_KEY"
  },

  "mcp_servers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."],
      "enabled": false
    }
  },

  "permissions": {
    "allow": ["bash(git *)", "bash(pytest*)"],
    "deny": ["bash(rm -rf*)"]
  },

  "model_roles": {
    "subagent": "gpt-4o-mini",
    "compact": "gpt-4o-mini",
    "review": "gpt-4o",
    "verdict": "gpt-4o",
    "plan": "gpt-4o"
  },

  "// pools": "Interchangeable models behind one name; usable anywhere an alias is.",
  "pools": {
```

```
    "frontier": ["gpt-4o", "claude-sonnet"]
  },

  "// routing": "Pick the model per request by how hard it looks (off by default).",
  "routing": {
    "enabled": false,
    "simple": "gpt-4o-mini",
    "medium": "gpt-4o",
    "hard": "frontier",
    "sticky": true,
    "rules": [{"match": "(?i)^deploy\\b", "level": "hard"}]
  },

  "hooks": {
    "PostToolUse": [
      {"matcher": "edit|multi_edit|write|apply_patch", "command": "git diff --stat"}
    ]
  },

  "sandbox": {
    "enabled": false,
    "network": false,
    "writable": ["~/.cache"]
  },

  "telemetry": {
    "jsonl": "~/.cache/coded/trace.jsonl",
    "otel": false
  },

  "injection_guard": "warn",
  "show_reasoning": false,
  "parallel_tools": true,

  "detect_external_agents": true,
  "external_agents": {
    "claude": {"command": ["claude", "-p", "{prompt}"], "timeout": 1800}
  },

  "auto_approve": false,
  "// verify": "After a turn that changed something, run the project's own checks and, with
model_roles.verify set, have a second model review the diff. Either complaint goes back to the
agent, up to max_rounds times. Both halves are off until configured.",
  "verify": {
    "command": null,
    "second_opinion": false,
    "max_rounds": 2,
    "timeout": 300
  },
  "auto_compact": true,
  "// auto_fallback": "When a model fails and its own fallbacks are exhausted (or it has none),
try the other configured models before giving up. This can send a request to a different
provider than the one you selected.",
  "auto_fallback": true,
  "// disabled_tools": "Tools never offered to the model, by name or pattern. Each tool's
definition is sent with every request, so dropping unused ones makes every step faster. Models
can disable more with their own \"disabled_tools\".",
  "disabled_tools": [],
  "compact_ratio": 0.8,
  "max_turns": 100,
  "auto_extend_turns": true,
```

```
  "stream": true
}
```

5. Developer Guide

Architecture, module map and conventions for working on coded itself. — from `CLAUDE.md`

Guidance for Claude Code (and other AI assistants) working in this repository.

Project overview

coded is a Claude Code-style terminal coding agent, written in Python, that works with any **OpenAI-compatible** chat-completions endpoint. It runs an agentic tool loop (read/write/edit files, search, run shell commands, spawn sub-agents) in an interactive REPL, supports multiple configurable models, a permission layer for risky actions, and an optional MCP client. See `README.md` for user-facing docs.

Tech stack

- **Language**: Python 3.10+
- **LLM access**: `openai` SDK, pointed at any OpenAI-compatible `base_url`. There is no hard dependency on OpenAI the company — the same client talks to Groq, Ollama, llama.cpp, etc.
- **Terminal UI**: `rich` (Markdown rendering, tables, live streaming) and `prompt_toolkit` (REPL input, history, completion).
- **MCP** (optional): `mcp` — guarded import; the app runs fine without it.
- **Tests**: `pytest`.
- Packaging via `pyproject.toml` (setuptools). Console script: `coded`.

Repository structure

```
coded/
├── __init__.py         # version
├── __main__.py         # `python -m coded`
├── cli.py              # argparse entry point, subcommand dispatch, wiring
├── config.py           # Config/ModelConfig, provider presets, layered loading
├── llm.py              # LLMClient: OpenAI-compatible streaming + tool calls
├── agent.py            # Agent: the agentic loop; sub-agent spawning
├── session.py          # Session: message history + usage/cost tracking
├── events.py           # EventLog: per-session structured facts (tool timings/errors, checks,
…)
├── permissions.py      # PermissionManager: approve/deny/always gate, accept-edits mode
├── trust.py            # project trust: which .coded/config.json keys need it, fingerprint
store
├── choice.py           # Chooser: arrow-key selector for permission prompts + ask_user (review
page)
├── confirm.py          # yes_no(): every yes/no outside a turn in the Chooser panel (keys or
lines)
├── notify.py           # Notifier: "Task completed/stopped: why" via desktop/voice/bell/line
├── lineedit.py         # LineEditor: the mid-turn input box's text, cursor, selection, lines
├── suggest.py          # command suggestions: grammar, live values, fuzzy, learning, next
steps
├── execstate.py        # Execution: running/tool/waiting/follow-up/interrupting/… apart from
input
├── tasks.py            # TaskManager: running sub-agents (progress, background, stop)
├── btw.py              # /btw: side question over a copy of the conversation (no tools)
├── prompts.py          # assembles main/sub-agent prompts + AGENTS.md/CODED.md/CLAUDE.md
hierarchy
├── prompt_files.py     # prompt-file loading/overrides/profiles + `coded prompts` CLI
├── limits.py           # per-model limits scaled by context_window; result/step caps
├── repo_overview.py    # repository overview for large-context models
├── repomap.py          # ranked symbol map (ast + regex, PageRank over references)
├── depgraph.py         # import graph (Python ast, JS/TS regex): dependents, impact, cycles
├── codehealth.py       # refactor hot spots: function metrics (ast), duplicate blocks, ranking
├── twin.py             # project twin: per-file parse cache (user cache dir) + /twin snapshot
├── lab.py              # coded lab / /lab: recurring failures in past sessions, approved
proposals
├── mission.py          # coded mission: unattended run in a worktree, restarts, report, notify
├── memory.py           # project memory store (./.coded/memory.json)
├── speed.py            # `coded models --speed`: first token, total, tok/s per model
├── verify.py           # post-turn gates: project checks + a second model
├── routing.py          # difficulty classifier + model pools (round-robin, health)
├── builtin_prompts/    # system, coder, tools, security, skills, planner, reviewer, debugger
(.md);
│                       # compact/ and full/ hold per-profile variants
├── repl.py             # interactive REPL and slash commands
├── tui.py              # full-screen Textual app (coded --tui), optional
├── sidebar.py          # the TUI's right panel: session, changed files, queue, sub-agents,
shells
├── dashboard.py        # /dashboard Command Center: agents, models, tools, checks, problems
├── timeline.py         # /timeline time-travel: turns, steps + outcomes, file states, restore
preview
├── ui.py               # rich-based rendering helpers (single Console)
├── skills.py           # skill discovery, frontmatter parsing, prompt section
├── review.py           # /review pipeline: reviewer→qa→security sub-agents
├── adversary.py        # /redteam, review --adversarial: read-only critic, evidence-checked
findings
├── builtin_skills/     # bundled roles: planner, reviewer, debugger (bodies from prompt
files), qa,
│                       # security, refactor, adversary
```

```
├── compaction.py       # context auto-compaction (summarize old messages)
├── sessions_store.py   # persist/restore sessions (--continue/--resume, /resume:
SessionKeeper)
├── checkpoints.py      # CheckpointManager: per-turn file snapshots for /undo and /rewind
├── transcript.py       # /context breakdown and /export Markdown
├── ignore.py           # .codedignore (gitignore syntax): rules_for(cwd), cached by mtime
├── diagnostics.py      # per-file checks after an edit (config "diagnostics"), off by default
├── styles.py           # output styles (/style): builtin_styles/*.md < user < project
├── builtin_styles/     # explanatory, concise, learning (.md, package-data)
├── statusline.py       # user status-line command (JSON on stdin, cached, background)
├── doctor.py           # `coded doctor`: requirements, config, keys, tools, --network probes
├── update_check.py     # daily background `git ls-remote` for git checkouts; notice at start
├── embeddings.py       # EmbeddingClient + embedding-model resolution
├── index.py            # CodeIndex: chunk/embed/store + cosine search
├── websearch.py        # web-search backends + HTML→text fetch
├── netguard.py         # which URLs are internal (loopback/LAN/metadata/file) for
web_fetch/browser
├── images.py           # image → base64 data URL for vision models; clipboard image grab
├── mentions.py         # `@path` mentions: expansion (Agent.run) + the REPL path completer
├── documents.py        # PDF/DOCX/XLSX/PPTX → text sections for the read tool
├── mcp_client.py       # optional MCP stdio client (background asyncio loop)
├── lsp.py              # LSP client: one language server per language, shared per process
├── hooks.py            # HookRunner: PreToolUse/PostToolUse/UserPromptSubmit/SessionStart/Stop
├── sandbox.py          # bwrap / sandbox-exec wrapping for bash (no Windows backend)
├── shells.py           # shell resolution (Git Bash/PowerShell/cmd), tree kill, UTF-8, rmtree
├── textio.py           # read/write text files preserving their line endings
├── background.py       # BackgroundShells: run_in_background processes
├── commands.py         # custom slash commands (.coded/commands/*.md)
├── telemetry.py        # Tracer: JSONL spans + optional OpenTelemetry
├── worktree.py         # git worktree isolation for task / external_agent
├── external_agents.py  # outside agent CLIs (claude, codex, gemini, aider, …)
├── acp.py              # `coded acp`: Agent Client Protocol server (stdio JSON-RPC)
├── mcp_server.py       # `coded mcp-serve`: coded as an MCP server
├── sources.py          # fetch dir / zip (safe extract) / git / GitHub shorthand into a temp
dir
├── plugins.py          # skill + plugin install, registry (installed.json), marketplaces,
│                       # plugin contributions merged into skills/commands/config
├── manage_cli.py       # `coded skill …` and `coded plugin …` subcommands
├── agent_import.py     # `coded import`: other agents' MCP servers + Claude Code commands
└── tools/
    ├── __init__.py     # build_registry(): assembles the tool suite
    ├── base.py         # Tool, ToolResult, ToolContext, ToolRegistry
    ├── files.py        # read, write, edit, ls, move, delete
    ├── patch.py        # multi_edit, apply_patch (Codex patch format)
    ├── notebook.py     # notebook_edit (replace/insert/delete a cell; read renders .ipynb)
    ├── browser.py      # browser (Playwright; opt-in "browser"): render, steps, console,
screenshot
    ├── todo.py         # todo_write (TodoStore), enter_plan_mode, exit_plan_mode
    ├── interact.py     # ask_user (PermissionManager.question), view_image (vision)
    ├── lsp.py          # lsp (opt-in "lsp"): definition/references/hover/symbols/diagnostics
    ├── external_agent.py  # external_agent (delegate to an outside CLI agent)
    ├── search.py       # glob, grep
    ├── repo_map.py     # repo_map (ranked outline of the codebase)
    ├── memory.py       # memory (remember/forget/list, gated per call)
    ├── semantic_search.py  # semantic_search (uses the embedded index)
    ├── shell.py        # bash (sandbox/background), bash_output, kill_shell
    ├── git.py          # git (read-only free, mutations gated)
    ├── github.py       # github (PR list/get/create, comment) via GITHUB_TOKEN
    ├── web.py          # web_search, web_fetch
    ├── task.py         # task (sub-agent delegation, run_in_background), task_output,
task_stop
    ├── skill.py        # skill (loads a SKILL.md body into context)
```

```
    └── mcp_tool.py     # wraps an MCP server tool; list_mcp_resources, read_mcp_resource
tests/
├── conftest.py         # FakeServer + EmbeddingsServer + RouteServer (all offline)
├── test_agent_e2e.py   # full agent loop (tool call, streaming, permission)
├── test_tui.py         # Textual TUI via Pilot (turn, permission modal, clear)
├── test_sidebar.py     # the side panel: file counts, sections, width, Ctrl+B
├── test_review.py      # /review pipeline + coded review verdict/exit code
├── test_tasks.py       # sub-agent tasks: activity, background, Ctrl+B, /tasks, Ctrl+O
├── test_queue_edit.py  # ↑ edits the newest queued message (REPL reader, dock, TUI)
├── test_suggest.py     # suggestion engine: matching, grammar, values, learning, context,
speed
├── test_suggest_tui.py # the TUI dropdown: keys only while open, click, unknown commands
├── test_persistent_input.py  # follow-ups reach the running turn, states, routing, interrupt
│                       # keeps input, history, TUI, a real REPL in a pty
├── test_cancel.py      # Agent.cancellable: Esc cuts a slow model call / kills a shell, TUI
├── test_line_editing.py  # mid-turn box: arrows/Home/End/words/selection/lines, POSIX pty +
│                       # Windows readers, echo redraw, drawn cursor, normal prompt untouched
├── test_btw.py         # /btw: session untouched, answered mid-turn, REPL + TUI
├── test_announce.py    # outcome + reason, channels, /notify, long-only, REPL/TUI/mission
├── test_confirm.py     # yes/no panel: line + key mode, defaults, Esc/Ctrl-C = no, no [y/N]
left
├── test_choice.py      # selector keys/lines, review page, permission feedback, REPL dock, TUI
├── test_trust.py       # project trust gate: what applies untrusted, remembered fingerprint,
CLI
├── test_events.py      # event log: tool outcomes, denials, timings, saved, bounded, thread-
safe
├── test_dashboard.py   # /dashboard render, REPL + mid-turn, TUI modal
├── test_slash_smoke.py # every slash command on a session full of "[x]"/"[/x]" text: no crash,
shown verbatim
├── test_timeline.py    # timeline turns, per-turn file changes, restore preview + rewind,
saved
├── test_adversary.py   # findings need file:line + trigger, read-only tools, model note, opt-
in
├── test_twin.py        # same answers with/without the cache, outside the repo, /twin changes
├── test_lab.py         # lab heuristics, per-folder sessions, one-by-one apply, loosening
flagged
├── test_turn_limit.py  # max_turns grows with progress, stops a loop, ceiling, hard-cap
switches
├── test_mission.py     # restarts/stops, open tasks, budget, branch + report, up-front
questions
├── test_session_autosave.py  # per-step autosave, atomic save, orphan tool calls, todos
├── test_resume.py      # /resume: list this folder's sessions, switch, keep saving under its
id
├── test_memory_bounds.py  # bounded output/file reads, timeout kills the process group
├── test_compaction.py  # context compaction trigger + tail preservation
├── test_permissions_sessions.py  # allow/deny rules, diff preview, sessions, json
├── test_checkpoints.py # checkpoint/undo + move/delete
├── test_git_tools.py   # git tool + github tool (fake API)
├── test_semantic_and_web.py  # index/search, web_fetch/search, image encoding
├── test_skills.py      # skill discovery, frontmatter, skill tool
├── test_agents_command.py  # `coded agents`: states, install hints per platform, --check
├── test_skill_sources.py  # real-world collections: links, nesting, YAML, prompt budget
├── test_tools.py       # tool + config unit tests
├── test_routing.py     # difficulty routing, pools, endpoint reuse, per-model cost
├── test_repomap.py     # symbol extraction, ranking, budgets, the repo_map tool
├── test_depgraph.py    # import resolution, impact + tests to run, cycles, repo_map operations
├── test_codehealth.py  # function metrics, elif-not-nesting, duplicates, ranking, /refactor
├── test_memory.py      # memory store, the permission gate, prompt injection
├── test_documents.py   # reading PDF/Word/Excel/PowerPoint, refusals, no-extra path
├── test_speed.py       # lazy SDK import, disabled_tools, compact tool schema, --speed
├── test_verify.py      # the verification loop: checks, second opinion, rounds
├── test_self_heal.py   # detected checks, ask-once approval, affected tests first, stall stop
```

```
├── test_auto_memory.py # fix notes, search, grouped recall, the fix-note offer after a healed
turn
├── test_model_chain.py # failover cooldown, sizing for a smaller fallback, stream retry,
│                       # per-model timeout, unknown-alias warnings
├── test_big_models.py  # scaled limits, caps, overflow retry, max_turns, profiles, repo
overview
├── test_windows.py     # shell resolution, line endings, UTF-8, shims (runs everywhere)
├── test_prompts.py     # prompt files: placement, override order, empty=remove, CLI
├── test_agent_import.py  # `coded import`: each agent's format, skips, secrets, ${VAR},
commands
├── test_new_tools.py  # ask_user, view_image, enter_plan_mode, MCP resources
├── test_lsp.py        # the lsp tool against tests/fake_lsp.py (a scripted server)
├── test_plugins.py     # sources, zip safety, skill/plugin install/update/remove,
│                       # Claude-format plugins, marketplaces, CLI
└── test_modern_features.py  # todo/plan, hooks, sandbox, AGENTS.md, patch, parallel,
                        # retry/fallback/roles, reasoning, shells, commands, tracing,
                        # worktrees, injection guard, external agents, ACP, MCP server
examples/skills/        # example SKILL.md packs + install guide
examples/plugins/       # example plugin (code-quality) + plugin layout guide
examples/github-actions/coded-review.yml  # PR review workflow (copy to .github/workflows)
examples/.coded-plugin/marketplace.json  # examples folder doubles as a marketplace
config.example.json     # sample configuration
docs/coded-documentation.pdf  # all docs in one PDF; rebuild: python docs/build_pdf.py
                        # (needs markdown + pygments, prints with Node Playwright's Chromium)
```

How it fits together

1. `cli.main()` parses args, loads `Config`, selects a `ModelConfig`, starts MCP (if configured), and builds an `Agent` via `agent.create_agent()`.
2. `Repl` (interactive) or `_run_print_mode` (one-shot) drives the agent.
3. `Agent.run(user_message)` appends to the `Session` and runs `_loop()`: call the model (`LLMClient.complete`) → if the response has `tool_calls`, execute each tool (through the permission gate) and append results → repeat until the model returns text with no tool calls, or `max_turns` is hit.
4. Tools implement `Tool.run(args, ctx) -> ToolResult`. `ToolContext` carries the cwd, the `PermissionManager`, the `Config`, and a `spawn_subagent` callable.
5. The `task` tool calls `spawn_subagent`, which builds a fresh `Agent` with a separate `Session` and a registry **without** the task tool (no infinite recursion), runs it non-interactively, and rolls usage back up to the parent.
6. **Skills** (`skills.py`): `discover_skills(cwd)` scans three locations — `coded/builtin_skills/` (bundled roles), `~/.config/coded/skills/`, and `./.coded/skills/` — for `<name>/SKILL.md`; a later location overrides an earlier one by name (so a project `planner` shadows the built-in). `create_agent` injects each skill's name+description into the system prompt (level 1) and registers the `skill` tool, which loads a skill's full body on demand (level 2). `/<skill> [task]` is `/skill <skill> [task]` (`skills.skill_message`; built-in and custom commands win a clash). Bundled files are reached with the normal read/bash tools (level 3). Sub-agents run without skills. Frontmatter is parsed by a tiny built-in YAML subset (no pyyaml dependency) — scalars, inline `[a, b]`, and block lists only. Built-in role skills (`builtin_skills/`) ship via `[tool.setuptools.package-data]`.

7. **Review pipeline** (`review.py`): `run_review(agent, target)` runs the `reviewer`, `qa`, and `security` role skills in sequence, each via `Agent.run_subagent` (isolated context), and aggregates their reports. Exposed two ways: the REPL `/review [target]` command, and the

non-interactive `coded review` subcommand for CI, which adds `synthesize_verdict()` (one model call → APPROVE/NEEDS_CHANGES) and maps NEEDS_CHANGES to exit code 1 (`--fail-on`, `--yolo`, `--output`, `--no-verdict`).

8. **Turn lifecycle** (`Agent.run`/`_loop`/`_run_tool`): `SessionStart` (first turn) and `UserPromptSubmit` hooks → plan-mode reminder appended → model call (`LLMClient`: retries, dropped unsupported params, `fallback` models) → tool calls, with consecutive parallel-safe calls run on a thread pool → per call: `PreToolUse` hook → plan-mode refusal → diff preview → permission gate (`auto_allowed` for sandboxed bash) → run → untrusted-output fencing/taint → nested `AGENTS.md` injection for file tools → `PostToolUse` hook. A final answer runs the `Stop` hook, which may push the agent to continue. 9a. **Which model answers a request** (`Agent._select_turn_model`, once per request, never per step): a forced model (`force_model`, a command's `model:`) > plan mode's `plan` role > difficulty routing (`routing.py`) > the main model. `switch_model()` pins the user's choice and turns routing off (`set_routing(True)` restores). `_turn_model` is the non-plan choice, so `exit_plan_mode` mid-turn returns to it. Routing refuses a target that lacks tools/vision or whose window the history no longer fits — never summarize to save money. Pools resolve only at named points (`Config.resolve_target`, `pick_from_pool`, `model_for_role`), never inside `get_model` on a hot path; `Agent._pool` makes the siblings fallbacks. Endpoints are process-wide (`llm.endpoint_for`), so health and learned-unsupported-params survive a switch. Cost accrues per answering model (`Session.record_usage(usage, model)`).

9. **Models per role**: `Config.model_for_role()` picks the model for `subagent`, `compact`, `review`, `verdict`, `plan` (`Agent.set_mode` swaps to the `plan` model via `use_model()` and back to `_main_model` on exit; `switch_model()` is the user's choice and sets `_main_model`); `Config.fallbacks_for()` resolves a model's `fallback` aliases for `LLMClient`.

10. **Skills & plugins install** (`plugins.py`): `coded skill|plugin …` fetch a source into a temp dir (`sources.Fetched`), validate it, show a summary and ask, then `install_tree` swaps it into `<home>/skills|plugins/<name>` and records the source in `<home>/installed.json` (`home` = `~/.config/coded` or `./.coded` with `--project`). Enabled plugins are merged at load time: `skill_search_dirs` (built-in < plugin < user < project), `command_dirs` (`/<plugin>:<cmd>`), and `load_config` → `apply_plugins_to_config` (hooks appended, MCP servers/outside agents added unless the user defines the name).

11. **Prompts are files** (`prompt_files.py`): `build_system_prompt` joins `system` + `coder` + `tools` + `security`, then an Environment section built in code, project instructions and extras. `build_subagent_prompt` uses a fixed sub-agent opener (tests rely on its first line) + `coder` + `tools` + `security`. A skill whose body is `<!-- coded:prompt NAME -->` loads prompt file NAME (planner/reviewer/debugger), so there is one source of truth.

12. **Per-model scaling** (`limits.py`): `limits_for(model)` scales the base (128k) limits by `context_window`, adds the per-result/per-step caps and the repo-overview budget, then applies the model's `limits` overrides. Each Agent (sub-agents too) computes its own and exposes them to tools as `ctx.extra["limits"]` (read with `get_limits(ctx)`), alongside `ctx.extra["model"]`. Prompt loading takes the model (`load_prompt(name, cwd, model)`): precedence built-in < built-in profile < user < project < model `prompts_dir`. `Agent.refresh_system_prompt()` rebuilds messages[0] (called by `create_agent` and `switch_model`).

Key conventions

- **Adding a tool**: subclass `Tool` in `coded/tools/`, set `name`, `description`, `parameters` (JSON Schema), and implement `run()`. Register it in `tools/__init__.build_registry()`. Set `requires_permission = True` for anything that mutates the filesystem or runs commands, and override `permission_detail()` to describe the action shown in the approval prompt.
- **Tool flags**: `parallel_safe = True` only for read-only tools with no permission prompt (or override `is_parallel_safe(args, ctx)`, as `task` does for worktree isolation). `untrusted_output = True` for tools returning third-party content (web, GitHub, MCP) so it's fenced and taints the turn. `auto_allowed(args, ctx)` lets a call skip the prompt (sandboxed bash); deny rules, plan mode and the strict injection guard still apply.
- **Shared agent state reaches tools via `ToolContext.extra`**: `todos`, `shells`, `get_mode`/`set_mode`. Sub-agents share the parent's `PermissionManager`, tracer and shells, but not todos/skills/Stop hooks.
- **Asking the user** (`ask_user`, 1-4 questions with headers, option descriptions, `multi_select`): `PermissionManager.questions(list[choice.Question])` goes to the front-end's `set_questions` callback (one answer string per question; `[]` = cancelled; None = nobody can answer: `--print`, ACP, which tells the model to state an assumption and go on). A front-end that only set the older `set_question(text, options)` is asked each question in turn. Main agent only, like `enter_plan_mode`, which refuses when `permissions.interactive` is false: nobody could approve the plan, so a `--print`/CI run would stay read-only.
- **Choice prompts** (`choice.py`): permission prompts ("Do you want to proceed?" → Yes / Yes, don't ask again / No, and tell coded what to do differently) and `ask_user` are one pure state machine, `Chooser`, fed key names (`key()`) or typed lines (`line()`) and drawn by `render()` — the same panel in the REPL and the TUI. Several questions get tabs and a "Review your answers → Submit" page. REPL: when the type-ahead reader can hand over keys (`TypeAhead.supports_keys`/`begin_keys`/`next_key`, the reader's `keys_mode`) and a `TurnDock` is up, the prompt replaces the dock's spinner and input box (`TurnDock.set_prompt`); otherwise it is printed with numbers and answered by a line (a follow-up typed at a permission prompt is requeued, never an answer). TUI: `PermissionModal`/`QuestionModal` (`ChoiceModal`). A "no" with text reaches the prompt callback as `"no: <text>"`; `request()` keeps it per thread and `Agent._run_tool` puts it in the denial (`take_feedback`). Questions outside a turn (project trust, mission up-front questions, plugin install, `/lab apply`, the setup wizard's yes/no) use `confirm.yes_no` — the same panel, driven by the type-ahead reader in key mode on a terminal, numbered lines otherwise (an injected `read` forces lines; `y`/`n`/empty = default work there). Never add a bare `[y/N]` prompt (`test_confirm` checks).
- **`view_image`** queues a path through `ctx.extra["attach_image"]`; after the step's tool results the agent adds one user message with the images (a `tool` message cannot carry one). `Config.disabled_tools_for` drops it for a model without `supports_vision`, so it is neither offered nor runnable. That message is not a turn boundary: code that looks for "the last user message" must skip it (`agent._is_tool_images`). MCP `list_mcp_resources`/`read_mcp_resource` are built only when a server reported resources at start (`MCPManager.resource_servers`).
- **Code intelligence** (`lsp.py`, the `lsp` tool, opt-in `config.lsp`, like `browser`, so the default request is unchanged): plain JSON-RPC over the server's stdio, no dependency. `get_manager(root, settings)` is process-wide (sub-agents get the tool and share servers); `stop_all` runs at exit. Files are re-sent when their mtime or size changes (`LSPClient.sync`), so answers match disk. Source lines of `.codedignore`d files are never printed. Server requests (`workspace/configuration`, …) are always answered or some servers wait forever. Positions are named by `line` + `symbol`; LSP columns are UTF-16 (`utf16_col`/`py_col`). A missing server or

unknown extension is a `ToolResult.error` pointing at grep/repo_map. Tests run against `tests/fake_lsp.py`, never a real server (one optional test skips).

- **Prompts from worker threads**: `PermissionManager.set_prompt(fn, threadsafe=True)` (TUI, ACP) allows tools that may prompt to run in parallel; prompts are serialized by a lock either way. `ask()` is a raw yes/no question (used by `exit_plan_mode`).
- **The system and role prompts live in `coded/builtin_prompts/*.md`.** (Short operational prompts — the sub-agent opener, `PLAN_MODE_REMINDER`, review-phase framing, compaction/verdict/`/init` prompts — are still in code.) Never `str.format` prompt files (users' overrides may contain braces); put dynamic values in code-built sections. New prompt files must be added to `PROMPT_NAMES`, and stay covered by the `builtin_prompts/*.md` package-data glob. Watch the size: coded targets small local models too.
- **A 128k model must behave exactly as before.** New size-dependent behaviour goes through `limits_for`; `test_128k_limits_equal_the_historical_constants` pins the base values. Tools never hard-code output sizes: use `get_limits(ctx)`. Every prompt profile must keep the untrusted-content and secrets rules (tested).
- **`coded agents`** (`external_agents.agent_statuses`/`install_hint`, `AGENT_INFO`): status of every preset/configured outside agent and the install command for missing presets. Install commands are **printed, never run** (same reason as "no install tool"). `which`/`platform` are injectable, so tests don't depend on what is installed; keep `AGENT_INFO` keys equal to `PRESETS` (tested), and check new install commands against the real registry before adding them.
- **Skill sources** (`sources.py`, `plugins.find_skills`): a fetched source has every symlink deleted unread (`strip_symlinks`); `Fetched.refuse_links_in` then refuses only an install whose own files held one (a skill dir, or a whole plugin/marketplace root). Skills are searched at the root, `skills/*/`, `*/`, then up to `_MAX_DEPTH` deep without entering a skill's own folders; duplicate names are an error. skills.sh page links and GitHub `tree/` links are parsed into GitHub sources. The frontmatter parser is indentation-based (block lists, one-level maps, `>`/`|` and plain multi-line scalars) — still no pyyaml. `skills_prompt_section(budget=limits.skills_chars)` shortens a big list; under budget it must be byte-identical to unbudgeted.
- **No install tool for the model.** Installing skills/plugins grants code execution (hooks run unprompted), so it stays a user-run CLI action. Never write plugin contributions into the user's config.json; they're merged at load time so `disable`/`remove` are clean. The install summary must list every hook and MCP command verbatim; non-TTY installs require `--yes`.
- **Tools must never raise into the loop**: return `ToolResult.error(...)` for expected failures. `Agent._run_tool` also catches unexpected exceptions and feeds them back to the model as text, so the agent can recover.
- **Modes** (`Agent.permission_mode`/`cycle_mode`, Shift+Tab): default → accept-edits (`PermissionManager.accept_edits`: `EDIT_KEYS` run unasked; deny rules, plan mode and the strict injection guard still apply) → plan. Mid-turn Shift+Tab (`typeahead.SHIFT_TAB`, `ESC [Z`; TUI while busy) calls `Agent.toggle_accept_edits`: accept-edits on/off only, never plan (that swaps the model, so it changes between turns). `ui.TurnDock(get_mode=…)` shows it. `Config.max_cost` is checked by `Agent.over_budget()` at the top of every loop step (main agent only), so a turn stops cleanly with a note.
- **Per-call permission** (git/github): a tool that is safe for some inputs and risky for others sets `requires_permission = False` and calls `ctx.permissions.request(...)` itself for the risky branch. A refusal's text comes from `PermissionManager.denial(what, next_step)` — it says *why* (user / rule / plan / unattended) via `peek_deny_reason`, which leaves the reason for the event log; never write "the user declined" by hand. Static-risk tools use the class flag instead.

- **Checkpoints**: mutating file tools call `ctx.checkpoints.record(path)` before writing so `/undo` works. `/rewind` uses `Agent._turn_marks` (one per `Agent.run` of the main agent: the user message **by identity**, the typed text and the checkpoint group id); `turn_history()` drops marks whose message compaction or `/clear` removed, and `CheckpointManager.undo_since(id)` undoes that group and every later one, newest first.
- **Paths outside the project ask first** (`Agent._outside_paths`, `_SCOPED_TOOLS`): a path that resolves (symlinks included) outside the cwd, `add_dirs` and the skill folders needs a yes — read-only tools get their own prompt ("Read file outside the project"), mutating tools reuse theirs with `request(always_ask=True)`, so accept-edits mode and sandbox auto-allow do not skip it. `--yolo`, "always" and allow rules (`read(/path/*)`) answer it.
- **Internal addresses ask first** (`netguard.py`, `tools.web.url_permission`): `web_fetch` and `browser` ask (once per host, `always_ask`) before a local file, localhost, a private/link-local/metadata address, and re-check every redirect (`fetch_url(allow=…)`) or browser request (`page.route`); a `file://` inside the project passes. Non-interactive runs refuse them.
- **`.codedignore`** is enforced in `Agent._run_tool` for path tools (`_PATH_TOOLS`, plus apply_patch's file headers via `_tool_paths`), in `search._walk` (grep/glob), `ls`, and `@mentions`/their completion. bash is not covered (documented). **Diagnostics** run after `_CHECKED_TOOLS` succeed, only when `config.diagnostics` is set, and are appended to the tool result. `add_dirs` reach the Environment section only when non-empty (the prompt is otherwise byte-identical), the sandbox's writable list and mentions.
- **Announcements** (`notify.py`): `Notifier(channels, after)` — `"notify"` is a channel list (`desktop`, `voice`, `bell`, `line`; `all`/`off`; the old single values still parse). `outcome(kind, reason, label, force)` only for work that ran `after` seconds (missions pass `force`); `outcome_of(agent, ok, error, interrupted)` reads the last request's events (interrupt, `max_turns` + reason, `over_budget`). REPL: `Repl._busy`'s finally (after the dock stops), TUI: `_run_agent`'s finally (its `liner` mounts a StatusNote from the worker), mission: end of `_run`. `/notify` (also mid-turn) calls `Notifier.set`; voice text goes through `_speakable` (letters, digits, light punctuation only).
- **Opt-in extras must not change the default request**: the `browser` tool is registered only with `config.browser`, an output style only adds a prompt section when set, and `statusline`/`update_check` never block (background threads; tests set `CODED_NO_UPDATE_CHECK=1` via an autouse fixture).
- **Usage** carries `cached_tokens` (`llm.usage_from` reads OpenAI's `prompt_tokens_details.cached_tokens` and DeepSeek's `prompt_cache_hit_tokens`); `session.usage_cost` prices them at `cached_input_cost` when a model sets it. `bash` changes are intentionally not tracked. New mutating file tools must record.
- **Network/optional-key tools** (github/web/semantic_search) must degrade to a clear `ToolResult.error` when the key/backend/index is missing — never crash. Their heavy imports are done inside `run()` to keep startup light.
- **Offline tests**: `conftest.py` provides `FakeServer` (chat), `EmbeddingsServer` (deterministic embeddings), and `RouteServer` (arbitrary HTTP for web/github). Keep new network features testable against these, not the real internet.
- **Return text, not exceptions, to the model.** Error strings should tell the model what to do differently.
- **Adding a skill** (docs/instruction pack, not code): create `./.coded/skills/<name>/SKILL.md` with `name` + `description` frontmatter. The `description` drives auto-invocation — say what it does and *when* to use it. See `examples/skills/`. No code changes needed. A skill shipped with the tool goes in `coded/builtin_skills/<name>/SKILL.md` (e.g. the planner/reviewer/qa/ security roles) and must be covered by the `package-data` glob in pyproject.

- **Adding a provider**: add an entry to `PROVIDERS` in `config.py` (base URL + API-key env var). Models reference it via `"provider": "<name>"`. Add local / keyless providers to `LOCAL_PROVIDERS` too. A model may instead be defined by just `base_url` + `model` (custom endpoint); `resolved_api_key()` falls back to a placeholder key so keyless endpoints work without config.
- **Model config fields** live on `ModelConfig`: `context_window`, `max_tokens`, `temperature`, `input_cost`/`output_cost` (per Mtok, used by `/cost`), `supports_tools`, `supports_vision`, `extra_headers`, and TLS (`verify_ssl`, `ca_bundle`). `config.example.json` is the catalogue and is validated by a test — keep it parseable and every model resolvable.
- **TLS**: both the LLM and embedding clients are built by `llm.build_openai_client`, which passes an `httpx.Client(verify=model.ssl_verify())` to the OpenAI SDK. `ssl_verify()` resolves `ca_bundle` → `$CODED_CA_BUNDLE` → `verify_ssl`. The `--ca-bundle`/`--insecure` CLI flags override the selected model; disabling verification warns. Route new endpoint clients through this helper.
- **Look**: purple theme (`ui.ACCENT`/`ACCENT_DEEP`/`ACCENT_SOFT`) and coded's own logo (`ui.LOGO`: a block-letter "coded" wordmark shaded light → deep purple by `logo_text`). Never hard-code colors or reuse another product's logo/branding. Tool calls display as `● Edit(app.py)` with results under `⎿` (`ui.tool_call_text` / `tool_result_text` / `diff_text` / `welcome`); the REPL and the TUI both render through these so they match. A new tool gets a readable title by adding it to `ui._TOOL_TITLES`.
- **Transcript spacing** (REPL): every block (tool call, assistant text, line-mode prompt) starts with `ui.gap()`, which prints a blank line only if the last row was not blank — `ui.console` is a `TranscriptConsole` that tracks this for every `print` (a Live's `Control` redraw does not count), so there is exactly one empty line between blocks. Never start a block with a bare `console.print()`. Assistant text gets a `●` with a hanging indent (`ui.answer_block`) only when the REPL turns it on (`set_answer_bullets`), so `-p` output stays plain.
- **Printed text is markup unless you say otherwise**: rich reads `[x]` in a `str` as a style tag (dropped silently) and raises `MarkupError` on a stray `[/x]`. `ui.info/warn/error/success` take **plain text** and escape it, so never put markup in their message. Anywhere else, text that is not a literal of ours (user messages, labels, commands, paths, descriptions, model or tool output, exception text) goes through `rich.markup.escape`, a `Text`, or `print(..., markup=False)` — table cells included. `tests/test_slash_smoke.py` runs every slash command on a session full of such text; add new commands to its `COMMANDS`.
- **UI**: use the helpers in `coded/ui.py` and its single shared `console`. Displayed tool output is truncated for readability; the **model** always receives the full tool result (truncation is display-only). Colors come from the palette constants (`ACCENT`, `MUTED`, …); `ui.thinking()` is the spinner used by the streaming path. The REPL (`repl.py`) adds a prompt_toolkit `bottom_toolbar` status bar, `AutoSuggestFromHistory` (Tab or → accepts it when no completion menu is open), and a shared `_PT_STYLE`.
- **Agent display is decoupled via events**: `Agent.on_event` (set by the TUI) receives structured events — `assistant_start/_delta/_end`, `reasoning_delta`, `tool_call`/`tool_result`/`diff` (with the tool-call `id`), `todos`, `status`. The TUI and the ACP server (`acp.py`) are both event consumers. When it's set, the agent emits instead of printing via `ui` (see `_emit`, `_show_tool_*`, `_call_with_events`). New display sites must route through these helpers, not call `ui` directly, so both front-ends stay in sync.
- **TUI** (`tui.py`, Textual, optional `[tui]` extra): the synchronous agent runs in a `@work(thread=True)` worker; events are marshalled to the UI thread with `call_from_thread`. The permission callback pushes a `PermissionModal` and blocks the worker on a `threading.Event` until the user answers. Guard the Textual import — the app must run fine without it installed.

The right-hand panel is drawn by `sidebar.render` (no Textual import, so it is testable alone) from agent state; changed-file counts come from the current checkpoint group (`sidebar.file_changes`) and read disk, so they are recomputed only after a tool result, a turn or `/undo` — never per token.

- **Streaming**: `LLMClient._consume_stream` accumulates text, reasoning (`reasoning_content`/`reasoning`) and tool-call deltas. Optional params (`stream_options`, `reasoning_effort`, …) that an endpoint rejects with a 400 are dropped and remembered per endpoint (`_OPTIONAL_PARAMS`). Keep this graceful-degradation habit for endpoints that implement only part of the OpenAI API. SDK retries are off (`max_retries=0`); `LLMClient._create` owns retry/backoff, and `complete()` owns fallback. A stream that fails is retried only if no text/reasoning was emitted yet (otherwise it would display twice).
- **Failures are explained, not dumped**: every error that ends a turn goes through `llm.explain_failure` (REPL, TUI and `--print`), which names the model and base_url and gives the next step. A 5xx gets one last attempt with the optional params (`_OPTIONAL_PARAMS`) removed, since some gateways answer an unimplemented param with a 500; `tools` is never dropped automatically. `coded models --check` probes a failing endpoint in stages (`llm.probe_stages`/`probe_hint`) to say which part of the request it refuses.
- **Startup and per-request size**: `openai`/`httpx` are lazy in `llm.py` (`_LazyModule`, warmed by `llm.prewarm()` on a thread from `cli.main`), so importing `coded.cli`/`coded.agent` must not import them — a test checks. Don't add top-level imports of heavy libraries; import inside the function. Tool definitions go out with every request: `Agent._tool_schema()` applies `Config.disabled_tools_for(model)` (fnmatch patterns, global + per model) and, for the compact prompt profile, `Tool.brief` plus argument descriptions cut to a sentence (`base._compact_parameters`). The standard profile's schema must stay byte-identical. A disabled tool is also refused in `_run_tool`. `coded models --speed` (`speed.py`) times the real turn shape.
- **Documents** (`documents.py`, through the `read` tool): PDF/DOCX/XLSX/PPTX become sections (pages, slides, sheets) of text lines, then go through the same offset/limit/budget code as text. The parsers are the `docs` extra and are imported lazily; a missing one is a `DocumentError` naming the install, and a binary file must **never** be decoded as text (a PDF is recognised by magic, and text with too many replacement characters is refused). Parsers raise anything, so `documents.load` converts every exception.
- **Repository map** (`repomap.py`, the `repo_map` tool): `ast` for Python plus conservative regexes elsewhere — an outline, not a parse tree, and never a new dependency. Ranking is PageRank over a file graph built from "names a symbol another file defines"; methods and names with more than `_MAX_DEFINERS` definers are excluded, or ubiquitous names decide the ranking. Everything is bounded (files, bytes, seconds, output) and nothing raises. It is a **tool**, not a prompt section: the system prompt stays names-only, and a 128k model must behave exactly as before.
- **Dependency map** (`depgraph.py`, `repo_map` with `operation`): `imports`, `dependents`, `impact` (reverse-transitive with depth, plus the test files among them: `affected_tests`) and `cycles`. Python imports via `ast` (absolute, relative, `src/`/`lib/` layouts, `from pkg import submodule`); imports inside a function are recorded in `DepGraph.lazy` — they count for impact but not for import-time cycles. JS/TS: relative specifiers only, resolved like a bundler (extensions, `index.*`, `.js`→`.ts`). Bounded, honours `.codedignore`, never raises. Same tool, not a new one: an omitted `operation` is the old outline.
- **Refactor finder** (`codehealth.py`, `repo_map` `operation: health`, the built-in `refactor` skill, REPL/TUI `/refactor [path]` → `refactor_prompt`): a plain scan, no model call — Python

functions over the thresholds (length, decision points, nesting where an `elif` is not a level, parameters without self/cls) and `DUP_WINDOW`-line copies in any code file (normalized, short and import lines skipped), ranked by excess × fan-in from `depgraph`. The edits go through the normal permission prompt, checkpoints and verify loop.

- **Project memory** (`memory.py`, the `memory` tool, `/memory`): short notes in `./.coded/memory.json`, injected by `build_system_prompt` as labelled data within `limits.memory_chars`. The model writes only the **project** store and only through `ctx.permissions.request` (per-call, like git — never `requires_permission`, which would also drag the turn through the verify gate), and not at all while `permissions.tainted`. Every write re-reads first, so a concurrent session's note is not clobbered. Sub-agents get no memory tool. An empty store must leave the system prompt byte-identical. Kinds: architecture, convention, decision, fix, gotcha, fact; recall is grouped in that order (newest first within a kind) and `search` (tool + `/memory search`) ranks by matched words. Automatic capture is one nudge, not a model call per turn: when the checks failed earlier in a turn and then pass, `_fix_note_nudge` asks the agent once to propose a `fix` note (approved like any note; never while tainted). Checks run with `PYTHONDONTWRITEBYTECODE=1` so a same-size fix within a second is not hidden by a stale .pyc.
- **Verification** (`verify.py`, `Agent._verify_turn`): after a turn that changed something, the project's `verify.command` and/or the `verify` role model gate the answer; a complaint goes back as a `[verify]` user message, bounded by `max_rounds`, then released with a visible note. The gate is "did a tool with `requires_permission` or `mutates` run", **not** the checkpoint group — `bash` edits are not checkpointed. Checkpoints only supply the diff. Never runs for sub-agents, plan mode or an unchanged turn; an unparseable verdict and a reviewer outage both release the turn. The second opinion needs `model_roles.verify` set explicitly: never auto-assign a critic.
- **Self-healing** (`verify.auto` or REPL/TUI `/heal`, `Agent.self_heal`): with no `verify.command`, `verify.detect_checks` finds the project's runner (npm test, cargo, go, pytest, make test). It runs repository code, so `Agent.auto_check_command` asks once per project + command (`trust.check_approved`/`approve_check`) and never runs it unapproved where nobody can answer. A detected pytest runs the affected tests first (`narrow_checks`, from `depgraph.affected_tests`); a failure identical to the previous round's (`same_failure`, timings/addresses normalized) stops the loop early with `stalled_note`.
- **Last-resort fallback** (`Config.auto_fallback`, on): when a model's own chain is exhausted, `Config.last_resort_models` offers the other configured models (same endpoint first, then the widest window) and `LLMClient._chain` drops the ones that cannot serve *this* request — no vision for an image, no tools for a tool turn, no window smaller than the conversation (never compact to survive someone else's outage). Declared fallbacks are never filtered. Each gets one attempt, a host that refuses a connection is skipped for its other models, and the error raised is the **primary's**, carrying `coded_also_tried`.
- **Model chain**: an endpoint that fails over is skipped for `fallback_cooldown` seconds (`_Endpoint.down_until`; cooling endpoints are tried last). `LLMClient.next_model()` is the model the next request goes to; `Agent._sync_sizing()` (each step, and on overflow) makes `self.limits` and compaction follow it via `_sized_model`, shrinking history when it is smaller. Use `_sized_model`, not `self.model`, for anything window-dependent in the loop. `Config.model_warnings()` reports unknown aliases.
- **Bounded memory**: never hold a command's output or a whole file of unknown size. `shells.run` spools output to temp files and keeps the head and tail past `max_bytes` (`MAX_CAPTURE_BYTES`); `grep` skips files over `GREP_MAX_FILE_BYTES`; `read` streams text over `READ_STREAM_BYTES`; background reads return at most `BG_READ_MAX_BYTES` of the newest output

(`tests/test_memory_bounds.py`). Unbounded reads let a sub-agent grow coded until the OS SIGKILLed it.

- **Event log** (`events.py`, `Session.events`, `Agent.events`): structured facts the conversation has no room for — each tool call (name, ms, error, denied with a `reason`: user | rule | plan | hook | unattended, from `PermissionManager.take_deny_reason` — only `user` is the person's choice — outside, sub-agent `task`), turns, verify pass/fail/stall, status notes (fallbacks, retries), overflow retries, max_turns, interrupts, answer length. Recorded by the `_run_tool` wrapper (the work is in `_run_tool_inner`) and the loop; sub-agents write into the parent's log. Bounded, locked, saved with the session (old sessions load without it) and **never** put in `session.messages`. `Agent.events` is a property that follows the agent's *current* session (`--resume` swaps it after construction); a sub-agent's is set to the parent's log. `/dashboard`, `/timeline` and `coded lab` read it.
- **Command Center** (`dashboard.render`, `/dashboard`): one plain render (no Textual import) from the session, its event log (`llm` events give the per-model cost), the task manager, shells, todos and verify state. REPL: a snapshot, also mid-turn via `TypeAhead(intercept=…)` (read-only); TUI: `DashboardModal`, refreshed every second, opened even while busy. Don't name a Screen method `_render` — it shadows Textual's own.
- **Time-travel** (`timeline.py`, `/timeline [N | restore N]`, `coded sessions show <id> [N]`): turns from `turn_history()` marks (live) or typed user messages (saved; coded's own `[verify]`/`[memory]` feedback is skipped); each step joins its tool result and its event (by tool-call `id`) for outcome and timing. A file's state *after* turn k = the backup of the next later checkpoint group that touched it, else the file on disk. Restore only previews (`restore_preview`), confirms and then calls `/rewind` — never a second undo path. Always say bash edits are not snapshotted and hidden reasoning is not stored.
- **Adversarial review** (`adversary.py`, `/redteam [target]`, `/review --adversarial`, `coded review --adversarial`): the built-in `adversary` skill in `run_subagent(read_only=True)` — the registry is built without `EDIT_KEYS` + delete/move/github (never by toggling the shared `plan_mode`); bash stays behind the normal prompt, and when the parent auto-approves (--yolo) the sub-agent gets its own gate with nobody to ask, so bash and git writes are refused. Fresh context with only the target; runs on `model_roles.review` and says when that is the main model. `parse` keeps a finding only with a known severity, a `file:line` that exists and a concrete trigger, and reports how many it dropped. Opt-in: `REVIEW_PHASES` and the CI exit codes are unchanged without the flag.
- **Project twin** (`twin.py`, `/twin`): per-file facts — raw imports (`depgraph.py_raw_imports`/`js_raw_imports`, resolved per build) and code-health findings + duplicate windows — keyed by `[size, mtime_ns]`; the whole file is versioned by `PARSER_VERSION` + coded's version. Stored in `$XDG_CACHE_HOME/coded/twin/<sha of realpath>.json`, never in the repo; atomic, corrupt = rebuilt, refreshed lazily (no watcher). `repo_map`'s dependency/health operations use it. Answers must be identical with and without it (tested); cached findings are stored as copies because ranking rescales scores. Tests get a private `XDG_CACHE_HOME` (autouse fixture).
- **Self-improvement lab** (`lab.py`, `coded lab [--apply]`, `/lab [apply]`): reads this folder's saved sessions (newest `MAX_SESSIONS`, size-capped) and their event logs; heuristics only, no model call — recurring tool errors (normalized), repeated denials, verify stalls, max_turns hits, identical repeated calls, fallback churn. A `Proposal` is shown as its exact change and applied only interactively, one at a time (no `--yes`; `--apply` refuses without a TTY); `loosens=True` ones get a warning. Writes go only to the project memory (gitignored) or `.coded/config.json` (via

`config_wizard`), never to committed instruction files. Denials count only with `reason == "user"`.

- **Step limit** (`Agent._loop`, `_extend_turns`, `stuck_reason`): at `max_turns` the limit grows by `max_turns` while the request makes progress, up to `_TURN_CEILING_FACTOR` × `max_turns`; `_recent_calls` (name, raw args, is_error) decides "stuck" (an identical call `_LOOP_REPEATS` times in `_LOOP_WINDOW`, or `_ERROR_RUN` failures in a row). `auto_extend_turns: false` or `--max-turns` (`max_turns_override`) is a hard cap. Events: `turns_extended`, then `max_turns` on the stop. Entries come from `call_record`: for `_POLL_TOOLS` (`bash_output`, `task_output`) the outcome is part of the key — a poll of something still running is never "the same call", a finished one with the same output again is. Those tools take `wait` (`BackgroundShells.wait`, `TaskEntry.done`; honours `ctx.extra["cancel"]`).
- **Missions** (`mission.py`, `coded mission`): every question is asked before the run (project trust, `trust.approve_check` for the detected checks, a `bash` allow rule); during it `permissions` has no prompt, so anything unapproved is refused and `ask_user` returns None. Edits run under `accept_edits`; the checks go in as `verify.command`. `run_mission` drives `Agent.run`; the first run is planning (`permissions.plan_mode` set by the mission, not `set_mode`, so no model swap or plan prompt) until a todo list exists (`PLAN_NUDGE` once, then `no_plan`). Then it decides from the run's events: an exception → backoff + continue, `max_turns` → continue unless its `reason` says not progressing, open todos → continue, `over_budget()` → stop; all bounded by `max_restarts`. It works in `worktree.create_worktree` (the project's `.coded/memory.json` is copied in), commits there at the end, and writes the report to `<repo>/.coded/missions/<id>.md`. `<id>.json` is the mission state (the up-front answers, session id, branch, worktree, status), saved at each run; `--resume` reloads it (`main_root` finds the main checkout from inside a worktree) and the session's todos, so it skips planning. Never pushes or merges. The REPL's `/mission …` calls `mission_main` with `--cwd` and the session's model; the TUI only points to it (the questions need a terminal). On a terminal `_steering` wraps `run_mission`: the type-ahead reader, the dock (`get_state` badge) and `Agent.followups`, so typed messages reach the mission at its next step; `STEERING_COMMANDS` answer at once, any other command goes back into the box; Esc = Ctrl-C (status interrupted, resumable). What was never delivered goes to the report (`undelivered`). Without a terminal (or `ui.is_quiet()`) nothing changes. Never name a local `escape` there: it shadows `rich.markup.escape`.
- **Sessions survive a crash**: `Agent.on_step` (main agent only; the CLI sets it to autosave) runs after the user message, each tool step and the final answer, so a kill loses at most one step; the REPL also saves after Ctrl-C. `SessionStore.save` writes atomically (temp + `os.replace`). The task list rides along as `Session.todos`: the CLI's autosave copies `agent.todos.items` into it, and `--continue`/`--resume` put it back into `agent.todos`. `sessions_store.close_tool_calls` gives unanswered tool calls an `INTERRUPTED_RESULT`: applied on load and at the start of every `Agent.run`, since endpoints reject a history with an orphan tool call. The saved id lives in a `SessionKeeper` (CLI autosave, `--resume`, and the REPL/TUI `/resume` via `resume_command`), so a session opened with `/resume` saves under its own id; the one left is saved first only if it has content. Checkpoints are not touched (`/undo` still reverts this run's edits). Tests get a private `CODED_SESSIONS_DIR` (autouse fixture).
- **Cross-platform**: anything that runs a command goes through `shells.run` / `command_argv` (POSIX keeps the historical `shell=True` string path; Windows gets an argv list for the resolved shell) and `popen_kwargs`/`kill_tree` for process groups. Capture output with `**text_kwargs()`, never bare `text=True` (Windows would decode with the ANSI code page). Editing a file goes through `textio.read_text`/`write_text` so line endings survive; never `Path.write_text` for user files. Platform logic takes `which`/`exists`/`env` arguments so it is testable on any OS

(`tests/test_windows.py`). The shell is settled once, by `load_config` → `shells.ensure_shell` (or `--shell`, which pins it), before plugin hook paths or the system prompt are built. Decoding a file is strict: edits refuse non-UTF-8 files (`textio.NOT_UTF8`) rather than rewriting them from a lossy read. Windows shells differ in more than quoting: PowerShell commands go through `-EncodedCommand` (and its CLIXML error stream is decoded back to text by `shells.clean_clixml`), cmd goes through subprocess's own `shell=True`, and `prompts._environment` tells the model which syntax to write (sub-agents included). An unknown `shell` name is a ConfigError, never a silent fallback. Arguments passed to a `.bat`/`.cmd` shim are re-parsed by cmd, so `external_agents.cmd_shim_risk` refuses prompts it could execute.

- **`coded import`** (`agent_import.py`): reads the MCP servers other agents configured (Claude Code, Claude Desktop, Cursor, VS Code, Gemini CLI, Codex TOML, Windsurf; paths in `candidate_files`, `home`/`platform`/`env` injectable), converts them with `convert`, prints every command verbatim and writes one config file's raw JSON through `config_wizard.read_file`/`write_file`, after a confirmation (non-TTY needs `--yes`). It never overwrites a name the file has, skips URL servers (stdio only), refuses literal `env` values for the project file and tightens the user file to 600 when it adds one. `mcp_client.server_params` expands `${VAR}`/`${VAR:-d}` in command, args and env, the form other agents' configs use for secrets; an unset one is named in `MCPManager.errors` (`unset_refs`). It also copies Claude Code commands (`discover_commands`: `~/.claude/commands` → the user commands dir, `./.claude/commands` → `./.coded/commands`, or all into the project with `--project`) byte for byte, except a frontmatter `model:` that is not a coded model, which is dropped (coded refuses an unknown one). Built-in slash-command names and existing files are skipped.
- **Interactive config** (`config_wizard.py`, `coded config setup`, REPL `/setup`): edits **one config file's raw JSON**, never a merged `Config` — otherwise another layer's models and plugin contributions would be written into the user's file. Unknown keys survive; writes are atomic and confirmed; API keys default to an env-var name and a literal key only ever reaches the user file (mode 600), never the project one. All prompting goes through `Prompter`, so the flow is testable without a TTY.
- **A long turn must stay legible and interruptible**: task progress goes through `ui.todo_progress`/`todo_bar`/`todo_title`/`current_todo` (panel, REPL toolbar and TUI status line all use them), and in the REPL `ui.todo_dock` pins the unfinished list under the spinner and the streaming view (transient, so the final answer frame drops it), and input typed mid-turn is routed (see *Persistent input*). During a REPL turn `ui.TurnDock` is the one rich Live (spinner, tasks, an input box showing `TypeAhead.typing`); the key reader's echo is off meanwhile, and `ui.thinking()`/`_call_streaming` draw into the dock (`ui.active_dock()`) instead of opening their own Live — rich allows only one. The dock never grows past the terminal (`ui._fit_height`): a Live taller than the screen is cut at the bottom, hiding the input box, so the spinner, todos, queue and input box always show and running sub-agents (`fit_running_tasks`: fewer steps, then headlines, then a count) and a streaming answer (newest lines) get the rest. Input typed mid-turn is queued by `typeahead.TypeAhead` (REPL) or `CodedTUI._queued` and run as the next message. The type-ahead reader owns stdin during a turn, so a prompt that needs an answer asks it via `TypeAhead.ask()` instead of reading stdin itself. Polling (never a blocking read) is what lets the reader stop with the turn instead of stealing the next prompt's first line. On POSIX the reader (`typeahead._PosixKeys`) puts the terminal in non-canonical, no-echo mode for the turn (signals stay on) so a lone **Esc** is seen, and `TypeAhead.stop()` must restore the terminal (`close()`). Both readers edit a `lineedit.LineEditor` (`reader.editor`; never a bare list): `_edit_sequence`/`_EDIT_SEQS` (POSIX, xterm `ESC [1;<mod>X` modifiers) and `_WIN_EDIT` + `_shift_held` (Windows) map keys to its moves/edits, `_EDIT_CTRL` the readline control keys;

unknown sequences are skipped whole. Home/End stay on the current line (Ctrl+Home/End for the whole text) — never jump. `\` +Enter / Alt+Enter insert a new line; ↑ on the first line is still `UP` (edit queued). The dock draws `TypeAhead.editing` (a `Snapshot`) with `ui.editor_text` (cursor block, selection, indented lines, styled runs not per-char spans). Ctrl+J cannot be a new line on POSIX (ICRNL makes Enter `\n` too). Esc stops the turn by sending this process SIGINT, so it behaves exactly as Ctrl-C. The dock's spinner shows `ui.spinner_word` (coded's own `SPINNER_WORDS`, rotating every few seconds from a random start) for a generic label and the turn's elapsed time; a specific `thinking(text)` label is shown as is.

- **Tasks** (`tasks.py`, `Agent.tasks`, `ctx.extra["tasks"]`): every sub-agent run is a `TaskEntry`; the sub-agent's `on_activity` (fired in `_show_tool_call`) records its steps. `task` with `run_in_background` runs `_work` on a thread and returns at once; with `TaskManager.can_detach` (set by the REPL for the turn when the type-ahead reader runs) a foreground task also runs on a worker so **Ctrl+B** (`entry.detach`) can return early, and Esc/ Ctrl-C stops it. A background thread calls `permissions.set_background`, so `request` denies instead of prompting and `ask`/`question` return None (the same once a task is stopped: its thread may still be finishing a step). Usage is added under `session.USAGE_LOCK` (inside `Session.record_usage`). Finished reports reach the agent as one user message (`report_message`): at the top of each step (`_deliver_background`), before a final answer (`_background_pending`, which also waits when `wait_for_background` — true except in the REPL), or by the REPL waking an idle prompt (`_WAKE` via `tasks.on_finish`). The REPL shows tasks in `ui.TurnDock` (`ui.running_tasks`), the status bar and `/tasks`; the TUI in the side panel's "Sub-agents" section (`sidebar.render`, refreshed on `task_*` events and by `_tick` while one runs) and a `task_end` note in the transcript. ("Tasks" in the sidebar is the todo list.) Control keys mid-turn (`typeahead.CTRL_B/O/T`) go to `TypeAhead(keys=…)`; the POSIX reader clears IEXTEN so ^O/^T reach it. **Ctrl+O** flips `ui.set_expanded`: `ui.tool_result` then prints whole results and `show_last_cut` re-prints every result shortened this turn (`_cuts`, cleared by `reset_cuts` at each turn), each under its tool call. Folded output is 3 lines (`_MAX_RESULT_LINES`), blank runs squeezed and long lines cut to one terminal row; the TUI's `ToolResult` folds the same way and a click (or Ctrl+O, `action_toggle_output`, for all) expands it.
- **Command suggestions** (`suggest.py`): one `Engine` for the REPL prompt (`repl.SuggestCompleter`, threaded), the mid-turn dock (`MenuState`, drawn by `ui.suggestion_rows`) and the TUI (`#suggest` OptionList). Commands come from `SLASH_COMMANDS` (TUI: `tui_commands()`, the ones it runs), custom commands and skills; arguments from `GRAMMAR` (keep it in step with a command's handler) whose slots name live `PROVIDERS` (models, styles, sessions, turns, tasks, shells, memory, rules, paths …) — never a canned list of values. Matching is `fuzzy` (prefix > substring > letters in order > typos); ranking adds `UsageStore` (per project, `$CODED_USAGE_STORE` in tests via an autouse fixture) and `next_steps` (from the last request's events, `Agent.active_tools` and agent state; only commands in this front-end's registry). Ties keep the source's order. Providers are cached (`_TTL`), a failing one yields nothing, `suggest()` never raises. A taken row only edits the input (`Suggestion.apply`): Enter takes a row only after ↑/↓ picked it, else it sends; Esc closes the list (mid-turn: the reader's `_escape` — Esc stops the turn only with the list closed). REPL mouse is opt-in (`config.mouse`, a SAFE_KEY). Unknown commands get `Engine.correct` ("Did you mean"); the TUI no longer sends an unknown `/word` to the model (paths like `/etc/x` still go).
- **Persistent input / routing** (`Repl._busy`, `Agent.followups`, `execstate.py`): every REPL path that runs agent work goes through `Repl._busy(label, kind)` — the main turn and `_handle_slash` for `_AGENT_COMMANDS`, skills, custom commands, `/plan <task>`, `/image … <prompt>` (`kind="agent"`) and `_WORK_COMMANDS` (`kind="command"`) — so the type-ahead reader,

the dock and the execution state are up for all of it (before, those commands ran with no input at all). Typed messages are follow-ups: `Agent.followups` (set for the turn: `TypeAhead.take_followups` / `CodedTUI._take_followups`) is read by `_deliver_followups` at the top of each step and before a final answer (`followup_message`, `FOLLOWUP_HEADER`; event `followup`); slash commands stay queued and run after; `_READ_ONLY_MIDTURN` answer at once. What is left when the work ends goes to `Repl._backlog` and runs next; on an interrupt `_keep_after_interrupt` puts it back into the input instead (TUI: `_paused`, the next one into the field) — never dropped, never run unasked. `Execution` is separate from input: front-ends call `begin/end/interrupt/waiting()`, the rest (running tools via `Agent.active_tools`, pending follow-ups) is read live; `badge()` draws it (dock `get_state`, TUI status line, REPL toolbar after a request). TUI Esc: priority `menu_close` while the dropdown is open, else `action_interrupt` → `agent.interrupt()`. With `Agent.cancellable` (TUI only; the REPL stops its main thread with SIGINT) that is immediate: the model call runs on a helper thread (`_call_cancellable`) and the wait raises `TurnCancelled` (a BaseException, so retry/fallback never catches it; the loop ends the turn with nothing half-added); stream callbacks (`_cancel_guard`) end the abandoned stream at its next chunk; `bash` gets `ctx.extra["cancel"]` → `shells.run(cancel=…)` kills the tree (`CommandCancelled`, the tool result says so); a foreground sub-agent task is stopped too. Without a cancel event `shells.run` is unchanged. Mid-turn ↑/↓ (`DOWN` from the reader on the last line) walk history after the queue, Ctrl+R searches it, Ctrl+L clears. `suggest.highlight` colors the typed line in the prompt (`InputLexer`) and the box (`editor_text(spans=…)`).
- **Queued messages** (`TypeAhead._queued`, a deque): ↑ mid-turn (`typeahead.UP`, from `ESC [A`/`ESC OA` or Windows `\xe0H`) runs `edit_last`, moving the newest one into the line being typed; Enter queues it again. Messages left from earlier turns are requeued into each new turn's reader so they stay visible (`ui.queued_list` in the dock) and editable. TUI: the `up` binding (`action_edit_queued`) does the same when the prompt has focus.
- **`/btw`** (`btw.py`): one `LLMClient.complete` on a fresh client (`agent._make_llm`) over `side_messages` — a copy of the session with an in-flight tool step dropped and orphans closed, plus `BTW_INSTRUCTION` and the question — with the agent's normal tool schema (valid history, same cached prefix); a reply that is only a tool call is asked again once with no tools. Never appended to the session; usage is recorded. Mid-turn it is taken out of the queue by `TypeAhead(intercept=…)` (REPL) or before the busy check (TUI) and answered on its own thread.
- **`@path` mentions** (`mentions.py`) are expanded in `Agent.run` (main agent only) so every front-end gets them: a mention starts the message or follows whitespace and must name something that exists (cwd or `add_dirs`), so emails and decorators are left alone. The original text is kept; the files follow in a `<mentioned-files>` block, images go to the image list for vision models.
- **Project trust** (`trust.py`, `config._merge_project`): a repository's `./.coded/config.json` is written by whoever wrote the repository. Only `trust.SAFE_KEYS` (output/model-choice settings), deny rules, a stricter `injection_guard` and models that just name a known provider apply at once; everything else (hooks, MCP servers, outside agents, endpoints/keys/headers, allow rules, auto-approve, sandbox, shell, statusline, verify, lsp, …) and project-scope plugins apply only to a trusted project. Trust is stored per realpath with a fingerprint of those settings (`~/.config/coded/trusted.json`, `$CODED_TRUST_STORE` in tests), so a changed hook asks again. The REPL/TUI start asks once (`cli._ask_project_trust`, every command verbatim); `-p`, `review`, `commit`, ACP and `mcp-serve` stay untrusted with a stderr note unless `--trust-project` / `CODED_TRUST_PROJECT=1`. A new config key is sensitive unless added to `SAFE_KEYS` on purpose. Permission rule lists add up across layers (a later file can never drop a deny rule).

- **Config is layered**: presets → user file → project file → CLI flags. Don't hard-code paths or keys; go through `config.py`. Dict-valued settings (`hooks`, `sandbox`, `telemetry`, `external_agents`, `model_roles`) merge key-by-key across layers.
- Follow the style already in the code: type hints, `from __future__ import annotations`, dataclasses for state, concise docstrings, minimal inline comments (only where they add real value).

Development workflow

Setup

```
pip install -e '.[dev]'        # editable install + pytest
pip install -e '.[mcp]'        # add MCP support
pip install -e '.[docs]'       # read PDF/Word/Excel/PowerPoint
```

Run

```
coded                          # REPL (needs a model configured or OPENAI_API_KEY)
coded -p "..."                 # one-shot
python -m coded                # equivalent entry point
```

To try it without real API keys, point `--base-url` at a local server, or see the fake server in `tests/conftest.py`.

Test

```
pytest                         # full suite
pytest tests/test_tools.py -q  # just the unit tests
```

Tests are **offline**: `tests/conftest.py` starts an in-process fake OpenAI-compatible server, so the end-to-end tests exercise the real `openai` SDK and the full agent loop without network access. When adding features, prefer extending this fake-server approach over mocking internals.

Lint / format

No linter/formatter is configured yet. Match the surrounding style. If you add one (e.g. `ruff`, `black`), record the commands here.

Gotchas

- **argparse + free-form prompt**: the top-level parser has a `nargs="*"` positional `prompt`, which conflicts with argparse subparsers. Subcommands (`config`, `models`, `agents`, `doctor`, `index`, `review`, `sessions`, `commit`, `mission`, `import`, `skill`, `plugin`, `prompts`, `acp`, `mcp-serve`) are therefore dispatched manually by the first token in `cli.main()` — keep new subcommands on that path, not on `add_subparsers`.
- **ACP / MCP server modes keep stdout clean** (`ui.set_quiet(True)`): stdout carries only protocol messages.
- **Sandbox `/tmp` is private** (bwrap `--tmpfs /tmp`): writes there vanish after the command. The working directory is bind-mounted writable, so tests using `tmp_path` as cwd still work.

- **Parallel batches display after they finish**: the parallel path calls `_run_tool(tc, display=False)` and then shows calls/results in order; keep new display code inside `_run_tool`'s `display` guard.
- **MCP runs on a background thread** with its own asyncio loop (`mcp_client.MCPManager`); calls into it are made with `run_coroutine_threadsafe`. Always `stop()` it (the CLI does so in a `finally`).
- **Permission default is deny** when non-interactive and not `--yolo`. Don't change this to "allow" — it's the safe default that keeps `--print` mode from running shell commands unattended. **Deny rules always win**, even over `--yolo`; per-tool `permission_target()` supplies the string matched by rules.
- **Compaction preserves tool pairing**: `compaction.py` only cuts the history at a `user`-role boundary, so an assistant `tool_calls` message is never split from its `tool` results (the API rejects that). Keep that invariant.
- **JSON print mode** (`--output-format json`) must keep **stdout clean**: `ui.set_quiet(True)` routes all log helpers to stderr, and the agent is forced non-verbose/non-streaming. Only the final JSON goes to stdout.

Git & branching

- Develop on the branch assigned to your session.
- Clear, descriptive commit messages. Do not open a pull request unless asked.
- Never commit real API keys. Project-level config lives in `./.coded/`, which is gitignored.

Releasing

Bump the version in `pyproject.toml` and `coded/__init__.py` (a test keeps them equal), add a `## <version>` section to `CHANGELOG.md`, merge to main, then push a tag `v<version>` on main. `.github/workflows/release.yml` checks the tag against the package version and creates the GitHub Release from that section.

Keeping this file accurate

Update this file whenever the structure, tooling, tool suite, config schema, or conventions change. Verify commands and paths against the actual repository before documenting them.

6. Changelog

What changed in each release. — from `CHANGELOG.md`

0.4.0

Added

- `coded mission "<goal>"`: an unattended run. It asks everything up front (trust, the project's checks, shell commands), works in its own git worktree and branch, plans with the task list, retries model outages, continues after the step limit or with tasks left open (`--max-restarts`), respects `--budget`, commits its changes on the branch and ends with a Markdown report in `.coded/missions/` plus a desktop notification or a POST to `--notify-url`.
- The `max_turns` event records why the run stopped (`reason`).
- Missions plan first: the first run may only read and must write the task list before any change is allowed. `coded mission --resume [id]` carries on with a mission that was killed (sleep, restart, closed terminal) and still writes the report; `coded mission --list` shows them.
- `/mission <goal>` in the REPL starts a mission with the session's model and folder (`/mission --resume`, `/mission --list`).
- `/resume [n | id]` (REPL and TUI): list this folder's saved conversations and switch to one without restarting; the one you leave is saved first, the one you open keeps saving under its own id.
- Command suggestions as you type `/`, in the REPL prompt, the box shown while the agent works and the TUI: subcommands, flags and live values (models, styles, sessions, requests, tasks, shells, memory notes, permission rules, files, `@paths`); fuzzy and typo-tolerant matching with the matched letters highlighted; ranked by what this project uses and by what the session points at next (failed checks → `/heal`, edits → `/review`, …). ↑/↓ choose, Tab or Enter takes, Esc closes; a suggestion never runs by itself. Unknown commands get "Did you mean …?". Mouse in the TUI; in the REPL with `"mouse": true`.
- Messages typed while the agent works reach the *running* agent at its next step (or before it would finish) instead of waiting for the turn to end; read-only commands (`/btw`, `/dashboard`, `/tasks`, `/todos`, `/cost`, `/context`, `/help`) answer at once, other commands run after.
- Execution state shown apart from the input: running, running tool, waiting for you, follow-up pending, interrupting, and how the last request ended (completed / failed / interrupted) — in the dock, the TUI status line and the REPL status bar.
- While the agent works: ↑/↓ walk earlier messages, Ctrl+R searches them, Ctrl+L clears the screen; the typed command, flags, values and `@paths` are highlighted (also in the prompt between turns).
- Missions show the input box while they work: type a message and it reaches the mission at its next step; `/btw`, `/dashboard`, `/todos`, `/cost`, `/help` answer at once; Esc stops it (resumable). The report lists what you told it.
- Long work says how it ended: "Task completed" or "Task stopped: " (you stopped it, an error, the step limit, not making progress, the spending limit) by desktop notification, voice, bell and/or a line in the transcript (`"notify": "desktop,voice,bell,line"` or `"all"`). `/notify on|off|all|test` switches it for the session; missions always announce.
- Esc stops a running turn in the TUI at once: a model reply that is still coming (or slow to start) is cut off and a running shell command is killed with its child processes, as well as a sub-agent

the turn waits on.
- `docs/coded-documentation.pdf`: all the documentation in one PDF (`python docs/build_pdf.py` rebuilds it).

Changed

- The REPL spinner cycles through coded's own words while the model works ("Tracing…", "Untangling…", …, a new one every few seconds) and shows how long the turn has run; specific labels ("Compacting…") still win.
- Every skill is also a slash command: `/prompt-master write a mail` is `/skill prompt-master write a mail` (REPL and TUI), and skills appear in slash-command completion. Built-in and custom commands win a name clash.
- The input box shown while the agent works is a real line editor: ←/→, Home/End, Delete, word jumps (Ctrl/Option+←/→), Shift-selection, Ctrl+A/E/K/U/W, and multi-line messages (`\`+Enter, Alt+Enter; Ctrl+Enter on Windows).
- A completed command ends with a space, ready for its arguments.
- The questions asked before work starts — trust this project, a mission's checks and shell commands, installing a skill or plugin, `/lab apply`, the setup wizard's yes/no — use the same panel as permission prompts (↑/↓, Enter, Esc = no) instead of a bare `[y/N]` line.
- The TUI no longer sends an unknown `/word` to the model; it suggests a correction (or points REPL-only commands to the REPL). Paths such as `/etc/hosts …` still go to the model.
- Project memory has an `architecture` kind (which module owns what, how the pieces connect), recalled first: `/memory add architecture: …` or the agent's `memory` tool.
- A long request no longer stops at `max_turns` (100) while it is still making progress: the limit grows by `max_turns` again, up to five times, and stops early only when the agent repeats the same tool call or keeps failing. The stop message says why and that `continue` picks up where it left off, and it is shown even without verbose output. `"auto_extend_turns": false` or an explicit `--max-turns` keeps the old hard cap.

Fixed

- A refused action said "The user declined/denied" even when nobody was asked (a mission, `--print`, CI) or a deny rule blocked it. The message now says which: refused because nobody could approve it, blocked by a rule, or denied by you. A plain `sleep` while a background shell runs gets a note pointing at `bash_output(wait=…)`.
- A long background job could stop a request or a mission as "not making progress": checking on it with `bash_output` looked like the same call repeated. Polling a shell or task that is still running now counts as progress, and `bash_output` / `task_output` take `wait` to block until there is new output (or the job ends) instead of spending a step per check.
- Arrow keys, Home/End and Delete did nothing in the input box shown while the agent works: it had no cursor and dropped every escape sequence.
- `/review`, `/redteam`, `/refactor`, `/init`, `/compact`, `/verify`, `/plan <task>`, `/image … <prompt>`, skills and custom commands ran with no input at all: keys echoed raw into the output, Esc did nothing and typing was lost. They now keep the input box like any request.
- Ctrl-C / Esc used to discard messages typed during the turn; they now go back into the input.
- With many sub-agents running, the input box was pushed off the screen (shown as "…"); the dock now fits the terminal.
- `/dashboard` crashed (`ValueError: too many values to unpack`) whenever the session had a task list.

- Text that looks like terminal markup no longer breaks the display. A request, task, shell command, memory note, command description or error message containing `[/x]` crashed `/help`, `/memory`, `/timeline`, `/dashboard`, `/checkpoints`, `/rewind`, `/shells`, `/tasks`, `/commands` and any status message; text such as `[name]` was silently dropped, so `/help` showed `/model .` instead of `/model [name]` and `pip install 'coded[mcp]'` lost its extra. Skill and plugin install summaries are now printed verbatim too.

0.3.0

Security

- **Project trust.** A repository's `.coded/config.json` (and project-scope plugins) no longer apply hooks, MCP servers, outside agents, model endpoints, allow rules, auto-approve, sandbox or similar settings until you trust the project; the REPL/TUI ask once and remember the answer until those settings change. `-p`, `coded review`, `coded commit`, ACP and `mcp-serve` ignore them (note on stderr) unless `--trust-project` / `CODED_TRUST_PROJECT=1`. **If you keep hooks or endpoints in a project config, you will be asked once.**
- Permission rule lists add up across config files; a later file can no longer drop a deny rule.
- Reading, listing or writing outside the project (or `--add-dir` folders) asks first, symlinks included; accept-edits mode does not skip it.
- `web_fetch` and `browser` ask before localhost, private/link-local/metadata addresses and `file://` outside the project, including after redirects.
- `coded commit` no longer auto-approves every tool (git only, with `--commit`, deny rules respected); `coded review` honours `injection_guard`.
- `apply_patch` refuses to move onto an existing file; a timed-out MCP call is cancelled. The review workflow installs coded from the base branch and no longer uses `--yolo`.

New

- `/heal` self-healing loop, `/refactor`, `repo_map` operations `imports`, `dependents`, `impact`, `cycles` and `health`, `fix` memory notes and `/memory search`.
- `/dashboard` Command Center, `/timeline` time-travel (and `coded sessions show`), `/redteam` and `review --adversarial`, `/twin` project twin, `coded lab` / `/lab` self-improvement lab, all fed by a per-session event log.
- Claude Code-style prompts: arrow-key permission prompt with "No, and tell coded what to do differently", `ask_user` with up to four questions and a review page; folded tool output (ctrl+o / click to expand); new wordmark.

0.2.0

Sub-agents as tasks

- Live progress for each running sub-agent: model, running time, tool uses and its latest steps, shown under the spinner; `⎿ Done (…)` when it ends.
- `task` takes `run_in_background`: it returns at once and the report arrives as a message when it finishes, while you keep chatting.
- **Ctrl+B** moves a running sub-agent to the background; **Ctrl+T** or `/tasks` lists sub-agents and background shells (`/tasks <id>`, `/tasks stop <id>`).
- New tools for the model: `task_output` (check a task) and `task_stop`.

- A background task never stops to ask for permission; it reports what it could not do.
- TUI: a **Sub-agents** section in the side panel.

While a turn runs

- An input box stays at the bottom of the screen during a turn, showing what you type; Enter queues the message.
- ↑ pulls the newest queued message back into the input to edit it.
- **Shift+Tab** turns accept-edits on or off mid-turn (from the next edit on).
- **Ctrl+O** expands or collapses long tool output.
- `/btw <question>`: a quick side question answered from the conversation, even mid-turn, without being added to the history.

Sessions

- The session is saved after every step, so a crash or kill loses at most one step; `--continue` / `--resume` pick it up.
- The task list (todos) is saved with the session and restored on resume.
- Tool calls left unanswered by an interrupted turn are closed, so the history stays valid for the endpoint.

Memory and processes

- Command output, `grep`, `read` and background-shell logs are bounded, so a large output can no longer grow coded until the OS kills it.
- A command that times out is stopped together with its child processes.

Fixes

- A sub-agent stopped with Esc no longer raises a permission prompt afterwards.
- `/btw` asks again without tools when the model replies with a tool call.
- Token and cost totals stay correct with background sub-agents running.